

Highlights

Understanding Token-level Topological Structures in Transformer-based Time Series Forecasting

Jianqi Zhang, Wenwen Qiang, Jingyao Wang, Jiahuan Zhou, Changwen Zheng, Hui Xiong

- We empirically find that existing Transformer-based TSF methods progressively degrade the original token-level topological structure (both positional and semantic) as network depth increases, thereby harming prediction performance.
- We propose a plug-and-play method, TEM, which explicitly preserves token-level topology in the intermediate layers of the network to improve prediction performance.
- Theoretically, we prove that TEM can tighten the model's generalization bound. Empirically, we show that applying TEM to multiple Transformer-based TSF methods significantly improves their prediction performance.

Understanding Token-level Topological Structures in Transformer-based Time Series Forecasting

Jianqi Zhang^{a,b,1}, Wenwen Qiang^{a,b,*,1}, Jingyao Wang^{a,b}, Jiahuan Zhou^c, Changwen Zheng^{a,b} and Hui Xiong^d

^aUniversity of Chinese Academy of Sciences, Beijing, China

^bNational Key Laboratory of Space Integrated Information System, Institute of Software, Chinese Academy of Sciences, Beijing, China

^cWangxuan Institute of Computer Technology, Peking University, Beijing, China

^dHong Kong University of Science and Technology, Hong Kong, China

ARTICLE INFO

Keywords:

Time series forecasting

Transformer

generalization bound

bi-level optimization.

ABSTRACT

Transformer-based methods have achieved state-of-the-art performance in time series forecasting (TSF) by capturing positional and semantic topological relationships among input tokens. However, it remains unclear whether existing Transformers fully leverage the intrinsic topological structure among tokens throughout intermediate layers. Through empirical and theoretical analyses, we identify that current Transformer architectures progressively degrade the original positional and semantic topology of input tokens as the network deepens, thus limiting forecasting accuracy. Furthermore, our theoretical results demonstrate that explicitly enforcing preservation of these topological structures within intermediate layers can tighten generalization bounds, leading to improved forecasting performance. Motivated by these insights, we propose the Topology Enhancement Method (TEM), a novel Transformer-based TSF method that explicitly and adaptively preserves token-level topology. TEM consists of two core modules: 1) the Positional Topology Enhancement Module (PTEM), which injects learnable positional constraints to explicitly retain original positional topology; 2) the Semantic Topology Enhancement Module (STEM), which incorporates a learnable similarity matrix to preserve original semantic topology. To determine optimal injection weights adaptively, TEM employs a bi-level optimization strategy. The proposed TEM is a plug-and-play method that can be integrated with existing Transformer-based TSF methods. Extensive experiments demonstrate that integrating TEM with a variety of existing methods significantly improves their predictive performance, validating the effectiveness of explicitly preserving original token-level topology. Our code is publicly available at: <https://github.com/jlu-phyComputer/TEM>.

1. Introduction

Time series forecasting (TSF) is crucial in many practical applications, including weather prediction [1, 20, 25], energy planning [2, 38], and traffic flow forecasting [10, 24, 46]. Its goal is to predict future value trends over a certain historical observation. Recently, deep learning approaches (e.g., RNNs [41], CNNs [29], Transformer [45], and MLPs [28]) dominate TSF due to their ability to model complex, nonlinear patterns. Among these, Transformer-based models [51, 27, 30] leverage self-attention to capture both short-range and long-range dependencies in parallel, consistently achieving state-of-the-art (SOTA) performance on TSF benchmarks.

Essentially, Transformer-based TSF methods can be understood as learning from the underlying topological structures among tokens [58, 37, 30]. Specifically, tokens are obtained by dividing the input sequence into multiple subsequences, each corresponding to one token. The topological structures between these tokens generally fall into two categories. The first is positional topology, which refers to the positional relations between tokens (Definition 3.1). The second is semantic topology, which refers to the similarity of features between tokens (Definition 3.2). Positional topology is typically modeled using positional encoding (PE). At the input layer, the model injects PE into each token to provide explicit positional information, helping the model identify temporal patterns such as trends, thereby improving performance. For example, in the Weather dataset [30], if the model knows that “9 AM precedes 1 PM” and the temperature at 9 AM is lower, it can recognize an upward trend in temperature during that period. Semantic topology is

*Corresponding author

✉ jlu-zhangjianqi@163.com (J. Zhang); qiang.ww0922@gmail.com (W. Qiang); wangjingyao2023@iscas.ac.cn (J. Wang); jiahuanzhou@pku.edu.cn (J. Zhou); changwen@iscas.ac.cn (C. Zheng); xionghui@ust.hk (H. Xiong)

¹These authors contributed equally to this work.

commonly modeled through the attention mechanism, which computes pairwise token similarity to capture topological structure within the sequence. This capability enables the model to identify temporally significant patterns—such as periodicity or recurring trends—by attending to tokens with similar semantic features. For instance, in the Weather dataset, attention weights may highlight strong similarities in variables observed at 24-hour intervals, allowing the model to discern and utilize these periodic spans for more accurate forecasting.

Although recent strategies for modeling token-level topology have substantially boosted the forecasting performance of Transformer-based TSF models, it remains unclear whether these methods fully leverage the intrinsic topological structures present at the token level. To clarify this, we revisit existing approaches through the perspective of structure preservation, a concept inspired by Locality Preserving Projection (LPP) [15, 16, 3, 48]. Structure preservation emphasizes maintaining the “original topological structure” among the input data—characterized by distances, similarities, and adjacency—during feature transformations. This practice produces models that are adept at capturing discriminative information in patterns, thereby improving downstream prediction performance [54, 55, 40]. In the context of Transformer-based TSF, we specifically define the original topological structure among the input data as the positional and semantic topology among input tokens in the input space, e.g., the **Original Positional Topology (OPT)** and the **Original Semantic Topology (OST)** (see Definition 3.3). A detailed analysis (see Section 3.2 for empirical evidence) indicates that, although existing Transformer-based TSF models implicitly capture certain topological structures, they do not explicitly enforce intermediate layers to preserve these structures. Consequently, successive transformations within these layers may unintentionally distort the information about the original topological structure among input tokens. Motivated by this observation, we hypothesize that explicitly enforcing structure preservation throughout the Transformer architecture could serve as a stronger inductive bias, thereby further enhancing TSF accuracy.

To empirically validate our hypothesis, we conduct a series of experiments (see Section 3.2 for details) to investigate the impact of enforcing structure preservation across the intermediate layers of Transformer models. Specifically, we explicitly incorporate the OPT and the OST into intermediate-layer features, aiming to help the model better retain these original topological structures. In addition, since both Rotary Position Embedding (RoPE) and Convolutional Position Embedding (CPE) can directly inject OPT into the intermediate layers of the network, we further extend our method by incorporating OST into both the input and intermediate layers based on these two position-encoding strategies, and conduct experiments to validate its effectiveness. The detailed procedure of this incorporation is presented in Section 3.2. After retraining the models under these conditions, we evaluate their predictive performance. As shown in Figures 1c and 1d, the experimental results demonstrate improved forecasting accuracy in most scenarios, although the magnitude of improvement varies across different forecasting tasks. To further understand the underlying causes of these empirical observations, we theoretically analyze how structure preservation influences the performance of Transformer-based TSF methods. Specifically, we prove two key findings: 1) explicitly maintaining the OPT and OST throughout deeper Transformer layers helps tighten the model’s generalization bound, thereby potentially improving its predictive generalization; 2) the extent of this improvement is inherently layer-dependent, explaining the observed variability in performance gains across different tasks. Consequently, we argue that adaptively enforcing the preservation of the original topological structure according to the characteristics of each layer in the model is essential for consistently achieving good forecasting performance.

To this end, we propose a novel Transformer-based method for TSF, termed Topology Enhancement Method (TEM). The core idea of TEM is to adaptively enforce the preservation of the original topological structure among input tokens, thus improving the predictive accuracy of Transformer-based models. Specifically, TEM consists of two primary modules: 1) Positional Topology Enhancement Module (PTM), which strengthens the preservation of OPT by injecting a learnable weighted positional encoding (PE) constraint into intermediate-layer features; 2) Semantic Topology Enhancement Module (STEM), which enhances the preservation of OST by incorporating a learnable weighted similarity matrix among input tokens into intermediate-layer representations. To ensure optimal topology preservation, we introduce a bi-level optimization strategy, enabling the modules to learn optimal injection weights adaptively. The proposed TEM is a plug-and-play method that can be easily integrated with existing Transformer-based TSF methods. Extensive experiments on various real-world datasets demonstrate the effectiveness and superiority of our proposed framework. The main contributions of this paper are summarized as follows:

- Through comprehensive empirical and theoretical analyses, we reveal two critical insights: 1) the original topological structure among input tokens is progressively weakened within intermediate layers of existing Transformer-based TSF models, potentially harming forecasting accuracy; 2) explicitly preserving the original

topological structures among input tokens throughout the Transformer's intermediate layers substantially enhances forecasting performance, building effectively upon existing Transformer-based TSF methods.

- Motivated by the above insights, we propose a novel Transformer-based approach, called Topology Enhancement Method (TEM), which adaptively preserves the original topological structure among input tokens. TEM consists of two modules: 1) the Positional Topology Enhancement Module (PTem), which injects a learnable weighted PE to explicitly maintain OPT among tokens; and 2) the Semantic Topology Enhancement Module (STEM), which introduces a learnable weighted similarity matrix to explicitly preserve OST among tokens.
- To ensure optimal preservation of the original topological structure, we introduce a bi-level optimization strategy that adaptively learns optimal weights for OPT and OST injection. Extensive experimental evaluations on multiple real-world benchmark datasets verify the effectiveness and superiority of the proposed TEM.

2. Related Work

Transformer-based TSF methods: Transformer-based methods are one of the mainstream approaches for TSF. Currently, many improvements have been made to these methods. Some methods [50, 59, 26] combine the Fourier transform with the Transformer to leverage the periodicity of time series. GCformer [57] integrates convolution with the Transformer. Others [9, 44] enhance the attention mechanism to improve TSF performance. [14] enhances the Transformer's forecasting performance through residual regularization (PRReg). PatchTST [37] embeds multiple adjacent time points into one token to address the insufficient temporal information of a single token. Crossformer [56] and DSformer [52] utilize both temporal and variable tokens for TSF. iTransformer [30] shows that using only variable tokens can yield good results. More details about temporal/variable tokens' details can be found in Appendix C. Regardless of how these methods are designed, they all rely on the positional/semantic topology to assist the model in making predictions.

Positional & semantic topology in transformer-based TSF methods: Regarding positional topology (see Definition 3.1 for details), existing studies mainly enhance a model's ability to capture such topological relations by designing new positional encodings (PE). There is extensive work on PE in other domains, such as natural language processing (NLP) and vision. [43] proposes Rotary Position Embedding (RoPE) to improve Transformer performance in NLP. [42] uses relative positional encoding for NLP tasks. [21] decouples token series from PE and explores a new way to inject PE. [4] and [31] propose Convolutional Position Embedding (CPE) and relative positional encoding, respectively, to boost Transformer performance in vision. Inspired by these above works, the researchers in TSF have also proposed a series of methods. [18] uses a discrete wavelet transform to generate PE and injects it at the input layer. [18] proposes continuous PE, injected at the input layer. [33] introduces a learnable PE, injected at the input layer. [32] injects a fixed relative PE at every layer. Of course, beyond these TSF-specific designs, the most commonly used choices in TSF remain sinusoidal PE and learnable PE [19]. For semantic topology (see Definition 3.2 for details), recent works primarily strengthen the semantics encoded in tokens by improving tokenization schemes [37, 30] or employing multiple types of tokens [56, 52]. This enables the attention mechanism to better capture semantic topological relations. Unlike the above methods, which focus on only one topology, this paper jointly considers both positional and semantic topologies in transformer-based TSF methods, leading to improved forecasting performance.

Structure Preservation in TSF: Building on the works in Locality Preserving Projections (LPP) [15, 16, 3, 48], preserving the original topological structure of input data during feature mapping has a positive effect on improving a model's predictive performance. The positional and semantic topologies discussed above can be regarded as forms of the input data's original topology. For preserving positional topology in TSF, [32] can achieve this preservation by injecting fixed relative PE into every layer. Moreover, applying positional encodings that are injected at each layer from other domains (such as RoPE [43] and CPE [4]) to Transformer-based TSF methods can also maintain positional topology. However, these approaches do not finely control the amount of positional topology injected at each layer. This may result in too little injection, yielding limited predictive gains, or too much injection, which can adversely affect predictions. Our experimental results in Subsection 3.2 further prove this. In TSF, research on preserving semantic topology remains scarce and warrants further exploration. This paper addresses the preservation of both positional and semantic topologies in Transformer-based TSF and proposes an adaptively fine-controlled injection mechanism, aiming to inject the optimal amount of positional and semantic topology to further enhance predictive performance.

Notably, this paper is the first in the TSF field to adaptively preserve and control both positional and semantic topology at the intermediate layers. Although methods such as RoPE [43], CPE [4], and GL-PE [32] preserve positional

topology by injecting PE into intermediate layers, they lack fine-grained control over the injection strength. As a result, overly weak injection may yield limited gains, whereas overly strong injection can interfere with prediction and degrade performance. Our experiment (see Figure 11) further demonstrates that under the same experimental conditions as our method, the predictive gains of these methods are significantly lower than ours. This indicates that fine-grained regulation of the amount of topology injected is crucial for improving prediction. In this work, we not only adaptively preserve and control positional and semantic topology at the intermediate layers, aiming to maximize the benefits of injecting topological information, but also provide a theoretical justification for the necessity of finely tuning the amount of topological information injected at each layer.

3. Problem Analysis and Motivation

In this section, we first introduce the important definitions and notations related to this work. Then, through experiments and theoretical analysis, we investigate the ability of existing models to preserve the original topological structure of input tokens in intermediate features, as well as their impact on predictive performance, which provides important insights for the design of our method.

3.1. Definition and Notation

Time Series Forecasting: Given a historical time series composed of observations $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\} \in \mathbb{R}^{T \times N}$, the goal of time series forecasting (TSF) is to predict future observations $Y = \{\mathbf{x}_{T+1}, \mathbf{x}_{T+2}, \dots, \mathbf{x}_{T+S}\} \in \mathbb{R}^{S \times N}$. Here, T is the lookback window length (number of historical timestamps), and S is the forecast horizon (number of future timestamps to predict). Specifically, each $\mathbf{x}_i \in \mathbb{R}^N$ represents the multivariate observation vector at timestamp i , consisting of values from N variables.

Tokenizer: In Transformer-based TSF, the input time series is first segmented into a sequence of tokens, which are then processed through a stack of network layers to generate the final forecast. We refer to the outputs at each intermediate layer as intermediate-layer token features. Various tokenization schemes have been proposed—such as patch-level, channel-wise, and multiscale tokens—each designed to capture different structural or semantic aspects of the input sequence. A comprehensive overview of these token types is provided in Appendix C.

Topological Structures: As discussed in the introduction, the topological structures between tokens consist of two categories: positional topology and semantic topology. Positional topology can help the model identify temporal patterns such as trends, while semantic topology enables the model to discover key signals that share similar characteristics (e.g., periodicity). Formal definitions of these two topologies are provided below.

Definition 3.1 (Positional Topology). Let N_t be the number of tokens and $\mathcal{P} = \{1, \dots, N_t\}$ be the index set of tokens' positions. The *positional topology* is presented as:

$$\mathcal{G} = \{p(k) \mid k \in \mathcal{P}, p \in \Upsilon\}, \quad (1)$$

where Υ denotes the class of functions used to generate positional encodings (PE).

In Transformer-based TSF methods, the input token is first converted into intermediate-layer token features through an embedding layer. Initially, these tokens features do not carry any positional information [45]. Positional information is modeled explicitly by adding PE to this tokens features [45]. Since the number of tokens typically remains constant across Transformer layers, the positional information associated with each token is expected to stay consistent throughout the layers.

Definition 3.2 (Semantic Topology). Let the i -th intermediate-layer token feature output by the l -th layer be denoted as H_i^l , where $i \in \{1, \dots, N_t\}$ denotes the index of token feature, $l \in \{1, \dots, L\}$ denotes the index of layer. $l = 0$ means that the token feature equals to the input token. The *semantic topology* at the l -th layer is defined as the set of feature similarity among these tokens:

$$\mathcal{S}^l = \left\{ \text{sim}(H_i^l, H_j^l) \mid i, j \in \{1, \dots, N_t\} \right\}, l \in \{0, 1, \dots, L\}, \quad (2)$$

where $\text{sim}(\cdot)$ is the similarity metric.

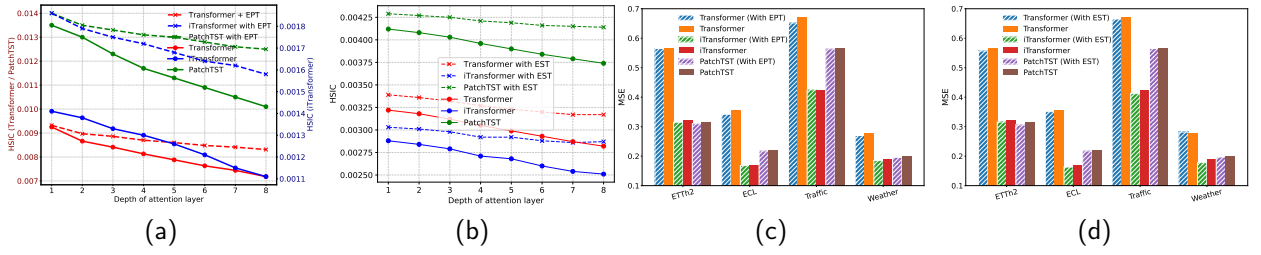


Figure 1: (a) Illustration of the change in the value of HSIC between the PE and the output feature of each encoder layer. (b) Illustration of the change in the value of HSIC between the similarity matrix of input tokens and the similarity matrix of output tokens of each encoder layer. Changes in model performance after enhancing positional topology of the input tokens (c) or semantic topology of the input tokens (d) in deep layers. EPT/EST stands for “enhanced positional/semantic topology of the input tokens”.

In Transformer-based TSF methods, the similarity between token features is naturally modeled by the Transformer’s attention mechanism. Specifically, the attention mechanism first computes pairwise token similarities by calculating the dot product between the Query and Key vectors. These similarities are then normalized through the softmax function to generate attention scores, which directly quantify the strength of the semantic dependencies among tokens. Thus, the attention mechanism inherently captures and the semantic topology in Transformer models.

Original Topological Structure among Input Tokens in TSF: Locality Preserving Projection (LPP) [15] defines the “original topological structure” of the input data based on two key aspects: the spatial distances and the feature similarities between neighboring samples in the input space. Analogously, in Transformer-based TSF, the original topological structure can be represented by the positional distances and feature similarities among input tokens in the input space. According to Definitions 3.1 and 3.2, these positional distances and feature similarities exactly correspond to the positional and semantic topology among tokens. Therefore, we define the original topological structure for Transformer-based TSF methods as follows:

Definition 3.3 (Original Topological Structure among Input Tokens of Transformer-based TSF). In the context of Transformer-based TSF, the original topological structure among input tokens, denoted as Θ , is composed of the positional topology and semantic topology among the input tokens in the input space, expressed as:

$$\Theta = \{\mathcal{G}, S^0\} \quad (3)$$

Thus, we name $\mathcal{G}$ as the Original Positional Topology (OPT) and S^0 as the Original Semantic Topology (OST).

By definition, positional topology is computed directly based on the positional relationships among tokens in the input space. Therefore, it is reasonable to adopt $\mathcal{G}$ as the OPT. Similarly, S^0 represents the semantic topology at the 0-th layer, which corresponds to the semantic relationships in the original input space. Hence, using S^0 as the OST is also well-justified.

3.2. Empirical Findings

In this subsection, we empirically verify two key findings: 1) the intermediate layers of existing Transformer-based TSF methods do not effectively preserve the original topological structure among input tokens; 2) explicitly enforcing the preservation of this original topological structure can further improve TSF performance.

First, we conduct experiments to evaluate how well Transformer-based models preserve the original topological structures among input tokens across different encoder layers. Specifically, we train three representative models—vanilla Transformer [45], PatchTST [37], and iTransformer [30]—on four standard datasets: ETTh2, ECL, Traffic, and Weather [30]. The vanilla Transformer and iTransformer adopt sinusoidal positional encoding (PE) [45], while PatchTST employs learnable PE [8]. In all experiments, the look-back and prediction lengths are set to 96, and the number of encoder layers is fixed to 8 for consistency and ease of analysis. To assess the preservation of the original local structure, we utilize the Hilbert-Schmidt Independence Criterion (HSIC), which measures the statistical dependence between two variables and is capable of capturing both linear and nonlinear relationships [13]. A higher HSIC value indicates a stronger dependency. In our context, a high dependence between 1) PE and the output features,

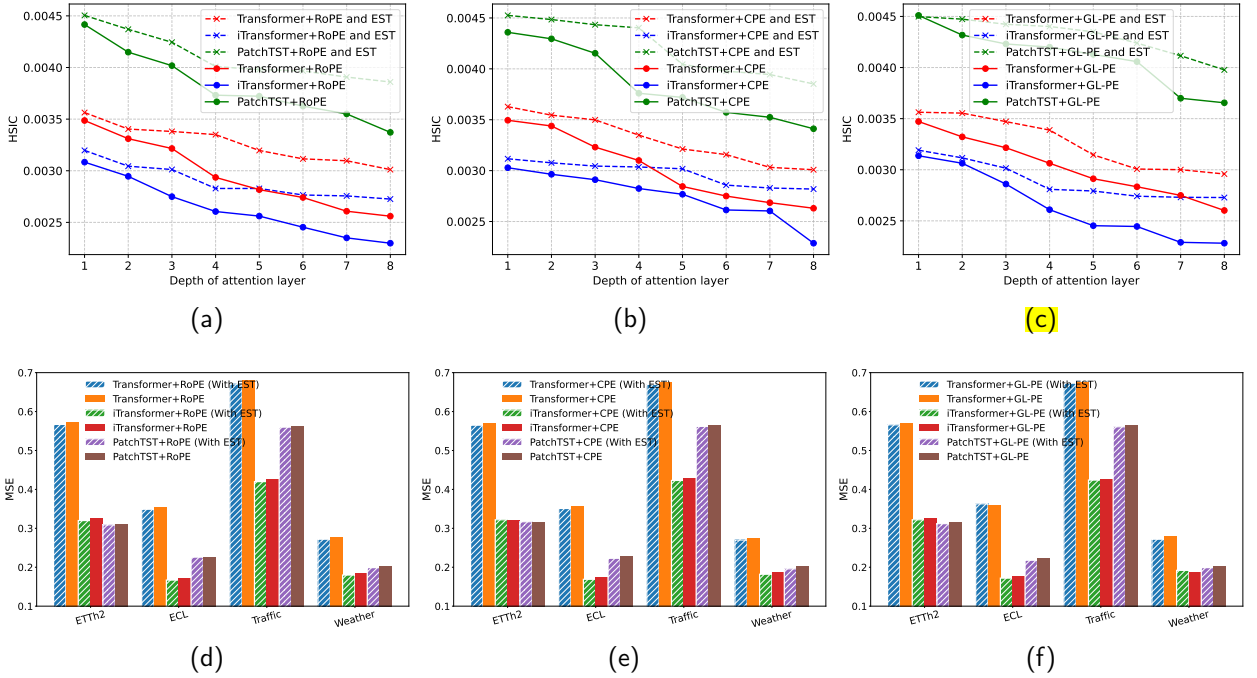


Figure 2: (a), (b), (c): Illustration of the change in the value of HSIC between the similarity matrix of input tokens and the similarity matrix of output tokens of each encoder layer. (d), (e), (f): Changes in model performance after enhancing the semantic topology of the input tokens. EST stands for “enhanced semantic topology of the input tokens”. Each row from left to right shows the results of the baseline methods using **RoPE** [43], **CPE** [4], and **GL-PE** [32].

and 2) the input-token similarity matrix and the corresponding output-token similarity matrix at each layer, suggests that the model retains the OPT and OST, respectively. Therefore, **HSIC serves as a reasonable proxy for evaluating the extent to which original local structures are preserved in intermediate layers.** The detailed computation process of HSIC is provided in Appendix I.2. Experimental results show that, as the network depth increases, the HSIC values for both OPT preservation and OST preservation consistently decrease across all models (Figures 1a and 1b). This indicates that the preservation of original local structures weakens progressively with depth, highlighting a potential limitation in current Transformer-based TSF models.

Second, we conduct experiments to evaluate the impact of explicitly injecting the original topological structures among input tokens into the intermediate layers of Transformer models. The dataset and model configurations remain the same as those used in the first set of experiments. To enhance the preservation of OPT, we add the $\mathcal{G}$ directly to the output features of each Transformer layer. To enhance the preservation of OST, we incorporate the $\mathcal{S}^0$ into each layer’s attention mechanism. Specifically, we multiply this similarity matrix by a scalar and add it to the raw attention logits before the softmax operation. Please refer to Appendix D for further discussion. Then, we retrain and evaluate the above models accordingly. To further verify the generality of the phenomenon, in addition to the experiments above, we include three other PE to compare: RoPE [43], CPE [4], and GL-PE [32]. Because **these three methods add PE at every layer of the model, they naturally inject OPT into intermediate layers.** We examine how these three PEs affect the three Transformer baselines before and after injecting OST into the intermediate layers. The experimental results in Figures 1a, 1b, 2a, 2b, and 2c indicate that both strategies effectively slow the degradation of layer-wise HSIC values, suggesting stronger preservation of the original topological structure. Furthermore, the results in Figures 1c, 1d, 2d, 2e, and 2f also show that, regardless of the PE adopted by the models, injecting OPT or OST into intermediate layers improves predictive performance in most cases, although the magnitude of improvement varies across tasks.

These two experiments reveal two key observations: **1)** under existing strategies employed by Transformer-based TSF methods, the original topological structures among input token are progressively degraded across intermediate layers as the network deepens; **2)** explicitly enforcing the preservation of these structures can lead to improved

forecasting performance. However, the current findings remain heuristic, and the simple quantitative enhancement strategies applied in these experiments may result in limited and inconsistent gains. These limitations underscore the need for a theoretical understanding of how the original topological structures influence Transformer-based model performance.

3.3. Theoretical Analysis

To explain the empirical observations in Section 3.2, we provide a theoretical analysis of how preserving the OPT and OST affects the performance of Transformer-based TSF methods. Specifically, we first derive a generalization error bound for Transformer-based TSF models based on the Rademacher complexity of each Transformer layer. Then, to investigate the impact of structure preservation, we analyze how the degree to which the original topological structures are retained in intermediate representations influences the Rademacher complexity of each layer, and thus the overall generalization error bound of the model.

Without loss of generality, consider a training set $\mathcal{D} = \{(X_i, Y_i)\}_{i=1}^m$ for the TSF problem. This dataset can equivalently be viewed either as samples drawn from a β -mixing distribution or as segments extracted from an extended, finite-length time series. Here, the mixing coefficient $\beta(\cdot)$ is defined according to Eq.(1) in [35]. Furthermore, this extended time series can be partitioned into μ mutually independent segments, each with length a . In this context, we consider the values at the starting timestamps of each segment as realizations of one random variable and the values at the ending timestamps as realizations of another random variable. Consequently, the coefficient $\beta(\cdot)$ measures the dependence between these two random variables. In what follows, we present the generalization error bound for Transformer-based TSF methods.

Theorem 3.4. *Let $\mathcal{X}$ and $\mathcal{Y}$ denote the input and output spaces for TSF, and let $\mathcal{H}$ be a class of Transformer-based functions $h : \mathcal{X} \rightarrow \mathcal{Y}$. Assume a non-negative, ξ -Lipschitz loss function $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$, bounded above by $M > 0$. Given a training dataset $\mathcal{D} = (X_i, Y_i)_{i=1}^m$, with probability at least $1 - \delta$, the following generalization bound holds for any $h \in \mathcal{H}$:*

$$\mathbb{E}_{(X,Y) \sim \mathcal{X} \times \mathcal{Y}}[\ell(h(X), Y)] \leq \frac{1}{m} \sum_{i=1}^m [\ell(h(X_i), Y_i)] + \xi G([\mathfrak{R}_\mu(\mathcal{H}_s^1)], \dots, [\mathfrak{R}_\mu(\mathcal{H}_s^L)]) + M \sqrt{\frac{\log(2/\delta')}{2\mu}}, \quad (4)$$

where $\delta' = \delta - 4(\mu - 1)\beta(a)$, $\mathfrak{R}_\mu(\cdot)$ is the Rademacher complexity of a function class, $\mathcal{H}_s^i$ is a function class of the i -th Transformer layer, and $G(\cdot)$ is an aggregation function over the layer-wise complexities.

This theorem provides an upper bound on the generalization error of Transformer-based TSF methods under a β -mixing distribution [35]. The bound consists of three components: the empirical risk over the training dataset, an aggregation of layer-wise Rademacher complexities reflecting the Transformer structure, and a confidence term. Note that the value of $G(\cdot)$ decreases as the Rademacher complexity of any single layer decreases. The detailed proof of Theorem 3.4 is provided in Appendix H.1.

Next, we analyze the relationship between the Rademacher complexity of a single-layer Transformer in Theorem 3.4 and the original topological structures preserved in the model. We first formalize several key concepts, including the layer-wise positional topology distortion and the layer-wise semantic topology distortion.

Definition 3.5 (Layer-wise positional topology Distortion). Let $\mathcal{G}$ denote the OPT among the input tokens, $p(k)$ denote the positional encoding (PE) vector of the k -th position, $p_i(k)$ denote the position encoding of the k -th token perceived from OPT at layer i . The layer-wise positional topology distortion at layer i is defined as:

$$\Delta_{\mathcal{G}}^i = \frac{1}{N_t^2} \sum_{k=1}^{N_t} \sum_{j=1}^{N_t} \left| \|p(k) - p(j)\| - \|p_i(k) - p_i(j)\| \right|. \quad (5)$$

Definition 3.6 (Layer-wise Semantic Topology Distortion). Fix a similarity function $\text{sim}(\cdot) : \mathbb{R}^d \times \mathbb{R}^d \rightarrow [0, 1]$ (e.g. cosine similarity). For layer $i \in \{0, 1, \dots, L\}$, let $H_k^i \in \mathbb{R}^{d_i}$ be the representation of token k . The layer-wise semantic topology distortion at layer i is defined as:

$$\Delta_S^i = \frac{1}{N_t^2} \sum_{k=1}^{N_t} \sum_{j=1}^{N_t} \left| \text{sim}(H_k^0, H_j^0) - \text{sim}(H_k^i, H_j^i) \right|. \quad (6)$$

Layer-wise positional-topology distortion measures the average absolute error between the pairwise token distances in the original (when $i = 0$) and the retained positional topology in the layer i ; hence $\Delta_G^i = 0$ if the full model preserves all pairwise positional structures of the input space. Layer-wise semantic topology distortion quantifies how much the pairwise semantic relationships observed in the input space (when $i = 0$) are distorted after propagating to layer i . In particular, larger values of Δ_S^i indicate greater degradation of the original semantic topology.

Theorem 3.7. *Under the condition of Theorem 3.4, the upper bound of the Rademacher complexity of l -th layer of the Transformer is:*

$$\mathfrak{R}(\mathcal{H}_s^l) \leq \mathfrak{R}(\mathcal{H}_s^0) \prod_{i=1}^l L_i \sqrt{1 - \rho_i} = \mathfrak{R}(\mathcal{H}_s^0) \exp\left\{\frac{1}{2} \sum_{i=1}^l [\ln L_i + \ln(1 - \rho_i)]\right\} \quad (7)$$

where ρ_i is the extent to which the original topological structure among input tokens is preserved at the i -th layer, i.e., $\rho_i = 1 - (\alpha \Delta_G^i + \beta \Delta_S^i) \in [0, 1]$, $\alpha, \beta \in \mathbb{R}^+$, and L_i is the Lipschitz constant of the function class corresponding to the i -th Transformer layer.

This theorem shows that preserving the original topological structure at each transformer layer can effectively reduce the Rademacher complexity upper bound, thereby improving its generalization ability. Specifically, it introduces a layer-wise topology preservation score ρ_i , which quantifies how well the original positional and semantic structure is maintained. When ρ_i approaches 1, indicating better structural preservation, the corresponding term $\sqrt{1 - \rho_i}$ becomes smaller, reducing the layer's contribution to overall model complexity. Recursively applying this effect across layers yields a global complexity bound determined by all ρ_i . Since Rademacher complexity directly influences the generalization error bound (Theorem 3.4), better structural preservation leads to stronger generalization. The detailed proof is provided in Appendix H.2.

Based on the above results, we can conclude that an appropriate preservation of input token topology across layers leads to a tighter generalization bound. These results explain the empirical observations in Section 3.2, i.e., the benefits of enhancing structure preservation. Considering the differences in topology information perception at different layers of the network and the values of α and β vary across different layers, finding an effective way to adaptively preserve the original topological structure among input tokens is important for boosting model performance.

4. Method

Based on the above analysis, we propose a plug-and-play Topology Enhancement Method (TEM). It aims to adaptively strengthen the preservation of OPT and OST at each Transformer layer to improve generalization performance (demonstrated in Section 3.3). TEM consists of two components: 1) **Positional Topology Enhancement Module (PTEM)**, which reinforces the preservation of OPT by injecting weighted PE into each Transformer layer (Section 4.1); 2) **Semantic Topology Enhancement Module (STEM)**, which enhances the preservation of OST by adding a weighted similarity matrix of input tokens to each attention map (Section 4.2). To ensure accurate topology injection, we introduce a bi-level optimization process to optimize the proposed TEM (Section 4.3). The framework of TEM is shown in Figure 3, and the corresponding pseudo-code is provided in Section G.

4.1. Positional Topology Enhancement Module

In this subsection, we introduce the details of the PTEM, i.e., how it injects OPT into each layer of the model.

Let H^l denote the input of the $(l + 1)$ -th Transformer layer. Let the number of heads be n_h . H^l is evenly split into n_h parts, which are used as the inputs for the n_h heads, denoted as $H^{l,i}$ ($i = 1, \dots, n_h$). Generally, for the i -th attention head in the l -th layer, $H^{l,i}$ is projected into the Query, Key, and Value matrices through three distinct fully connected (FC) layers. The core idea of PTEM is to inject the OPT into this process. Specifically, before feeding $H^{l,i}$ into the FC layers, PTEM first generates three learnable scalar coefficients, denoted as $\Gamma_Q^{l,i}$, $\Gamma_K^{l,i}$, and $\Gamma_V^{l,i}$, corresponding to the Query, Key, and Value paths, respectively. Then, let $[\cdot]$ be the matrix operation, the modified input for each path is computed as:

$$H_j^{l,i} = H^{l,i} + \Gamma_j^{l,i} \times [\mathcal{G}], \Gamma_j^{l,i} \in \mathbb{R}^+, j \in \{Q, K, V\}. \quad (8)$$

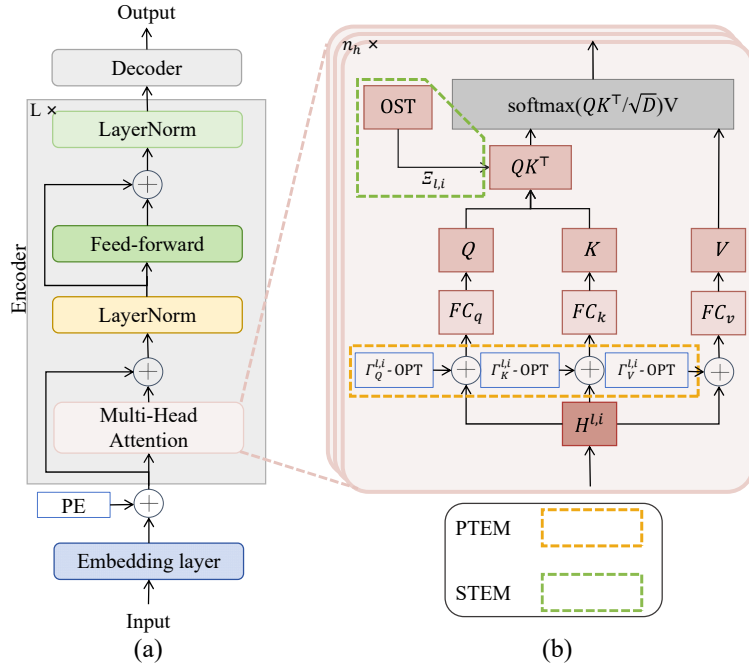


Figure 3: Overview of the Topology Enhancement Method (TEM). (a) Architecture diagram of the current popular Transformer-based TSF methods. (b) Illustration of the integration locations of PTEM and STEM within Transformer-based TSF methods.

Finally, the adjusted representations $H_Q^{l,i}$, $H_K^{l,i}$, and $H_V^{l,i}$ are passed through their respective FC layers in the i -th attention head of the l -th Transformer layer to generate the final Query, Key, and Value matrices. We represent the tensor Γ using the index notation $\Gamma = [l, i, j]$, where l , i , and j denote the Transformer layer index, attention head index, and projection type (Query, Key, or Value), respectively, and $\Gamma[l, i, j] = \Gamma_j^{l,i}$. This tensor can be interpreted as controlling the proportion of the OPT injected into each Transformer layer. In other words, by adjusting the values of elements in Γ , we can explicitly modulate the extent to which OPT is preserved in each layer of the Transformer. More details are provided in Appendix F.

4.2. Semantic Topology Enhancement Module

In this subsection, we introduce the details of the STEM, i.e., how it injects OST into each layer of the model.

In the i -th attention head of the l -th Transformer layer, a key computational step involves calculating the dot product between the Query and Key matrices, i.e., QK^T . Typically, each element in QK^T can be interpreted as the similarity between a pair of tokens, which aligns closely with the definition of the OST. Based on this observation, STEM enhances OST preservation within Transformer layers by injecting OST into QK^T directly, as follows:

$$QK^T = QK^T + \Xi_{l,i} \times [S^0], \quad \Xi_{l,i} \in \mathbb{R}^+, \quad (9)$$

where the $\text{sim}(\cdot, \cdot)$ in S^0 is computed directly using the inner product between vectors (i.e., as in QK^T), $\Xi_{l,i}$ denotes the injection strength of OST into the i -th attention head of the l -th Transformer layer. We represent the full OST injection matrix as $\Xi = [l, i]$, where each element $\Xi[l, i] = \Xi_{l,i}$ controls the proportion of OST injected at a specific layer and head. In this formulation, adjusting the values in Ξ enables fine-grained control over the extent to which OST is preserved throughout the network. More details are provided in Appendix F.

4.3. Overall Optimization

To ensure the accuracy of OPT and OST injection, we propose a **bi-level optimization mechanism** that jointly optimizes the TSF model (f_θ) and the parameters of PTEM and STEM modules (Γ, Ξ).

Specifically, during training, a batch (with N_b samples) is denoted as $\mathcal{D}_b = \{X_i, Y_i\}_{i=1}^{N_b}$. The whole objective of TEM is presented as:

$$\min_{\Gamma, \Xi, f_\theta} \mathcal{L}_{\text{mse}}(f_\theta, \Gamma, \Xi, \mathcal{D}_b), \quad \text{s.t.} \quad \min_{f_\theta} \mathcal{L}_{\text{mse}}(f_\theta, \Gamma, \Xi, \mathcal{D}_b), \quad (10)$$

where $\mathcal{L}_{\text{mse}}$ represents the loss that used in existing TSF methods, e.g., the MSE loss.

Since Eq.(10) formulates a multi-objective optimization problem, we adopt a bi-level optimization strategy to solve it. Specifically, for the inner-loop optimization, we fix Γ and Ξ , and update the model parameters f_θ using gradient descent:

$$f_\theta \leftarrow f_\theta - \eta_1 \nabla_{f_\theta} \mathcal{L}_{\text{mse}}(f_\theta, \Gamma, \Xi, \mathcal{D}_b), \quad (11)$$

where η_1 is the learning rate for the inner loop.

For the outer-loop optimization, we proceed based on the updated model from the inner loop. Let f_θ^1 denote the model parameters obtained after the inner-loop update. We then update Γ and Ξ via gradient descent as follows:

$$\{\Gamma, \Xi\} \leftarrow \{\Gamma, \Xi\} - \eta_2 \nabla_{\{\Gamma, \Xi\}} \mathcal{L}_{\text{mse}}(f_\theta^1, \Gamma, \Xi, \mathcal{D}_b) \quad (12)$$

where η_2 is the learning rate for the outer loop. It is important to note that the loss term $\mathcal{L}_{\text{mse}}(f_\theta^1, \Gamma, \Xi, \mathcal{D}_b)$ is evaluated using the parameters f_θ^1 obtained from the inner-loop optimization. Since f_θ^1 itself is a function of Γ and Ξ (as shown in Eq.(11)), the gradient $\nabla_{\{\Gamma, \Xi\}} \mathcal{L}_{\text{mse}}(f_\theta^1, \Gamma, \Xi, \mathcal{D}_b)$ involves a composite gradient, e.g., $\nabla_{\{\Gamma, \Xi\}} \mathcal{L}_{\text{mse}}(f_\theta^1, \Gamma, \Xi, \mathcal{D}_b) = \nabla_{f_\theta} \mathcal{L}_{\text{mse}}(f_\theta^1, \Gamma, \Xi, \mathcal{D}_b) \cdot \nabla_{\{\Gamma, \Xi\}} f_\theta^1$.

Explanation of the Learning Strategy of Γ and Ξ : It is important to emphasize that the learning of Γ and Ξ is not performed jointly with the learning of f_θ . Instead, we adopt a bi-level learning strategy in which Γ and Ξ are updated after the optimization of f_θ . The rationale behind this design is to accurately model the effect of preserving the original topological structure of the input space in intermediate layers on improving TSF performance. If we were to optimize Γ , Ξ , and f_θ simultaneously, e.g., by minimizing a joint loss $\min_{\Gamma, \Xi, f_\theta} \mathcal{L}_{\text{mse}}(f_\theta, \Gamma, \Xi, \mathcal{D}_b)$, it would be difficult to determine whether the improvement in the loss is actually due to the updates in Γ and Ξ , since the loss can already decrease by optimizing f_θ alone. In contrast, our bi-level strategy (as described in Eq.(10)) allows us to isolate the contribution of Γ and Ξ . That is, after minimizing the loss with respect to f_θ , we further update Γ and Ξ to reduce the loss again. This learning process directly reflects the principle that appropriately preserving the topological structure of the original input space in intermediate layers can further enhance TSF performance.

5. Experiments

In this section, we first introduce the experimental setup, followed by a comparison of our method with other approaches on eight benchmark datasets. Second, we conduct ablation studies to evaluate the contribution of each submodule. Third, we show the hyperparameter sensitivity of our method. Next, we analysis the computational complexity of our method. Finally, we present a visual comparison between our method and other methods. More experiments are provided in **Appendix I**.

5.1. Experimental Settings

Datasets: We use eight real-world datasets, including ECL, ETTh2 (h1,h2,m1,m2), Traffic, Weather in [30], and Solar-Energy dataset in [23]. We use the same train-validation-test split as in [30]. See Appendix E for details.

Baselines: We select ten classic time series forecasting (TSF) methods for comparison. These include: 1) Transformer-based TSF methods such as PatchTST [37], iTransformer [30], Transformer [45], Crossformer [56], DSformer [52], and PRReg CI(Transformer) [14]; and 2) other widely used TSF methods from different model families, including DLinear [53], SpareTSF [28], TIDE [7], and TimesNet [49].

Implementation Details: All models are trained on an NVIDIA 4090 GPU. The evaluation metrics used are Mean Squared Error (MSE) and Mean Absolute Error (MAE). The current mainstream Transformer-based methods primarily handle three types of tokens: temporal tokens, patch tokens, and variable tokens. These tokens are generated using different tokenization strategies, which are detailed in Appendix C. To verify the effectiveness of our method across different types of tokens, we apply the proposed Topology Enhancement Method (TEM) to three methods.

Table 1

Full MSE results for the TSF task. All baseline models' input length is 96, and the prediction lengths include {96, 192, 336, 720}. The results of our method are averaged over five random seeds, and the standard deviation across the five runs is reported after the “ $\pm$ ” symbol.

Models	PatchTST+TEM	PatchTST	iTransformer+TEM	iTransformer	Transformer+TEM	Transformer	Crossformer	TimesNet	DLinear	DSformer	SpareTSF	TIDE	PRReg	CI
Metric	(Ours) MSE	(2023) MSE	(Ours) MSE	(2024) MSE	(Ours) MSE	(2017) MSE	(2023) MSE	(2023) MSE	(2023) MSE	(2023) MSE	(2024) MSE	(2023) MSE	(2024) MSE	
ETT <h>1</h>	96	0.404 $\pm$ 0.002	0.414	0.380 $\pm$ 0.001	0.386	0.550 $\pm$ 0.002	0.611	0.423	0.384	0.386	0.401	0.374	0.479	0.409
	192	0.451 $\pm$ 0.000	0.460	0.435 $\pm$ 0.002	0.441	0.627 $\pm$ 0.000	0.697	0.471	0.436	0.437	0.441	0.419	0.525	0.454
	336	0.490 $\pm$ 0.001	0.501	0.478 $\pm$ 0.000	0.487	0.693 $\pm$ 0.001	0.770	0.570	0.491	0.481	0.486	0.463	0.565	0.514
	720	0.488 $\pm$ 0.001	0.500	0.495 $\pm$ 0.000	0.503	0.716 $\pm$ 0.000	0.796	0.653	0.521	0.519	0.493	0.477	0.594	0.530
	Avg	0.458	0.469	0.447	0.454	0.646	0.718	0.529	0.458	0.456	0.455	0.433	0.541	0.477
ETT <h>2</h>	96	0.295 $\pm$ 0.000	0.302	0.291 $\pm$ 0.002	0.297	0.423 $\pm$ 0.001	0.470	0.745	0.340	0.333	0.319	0.308	0.400	0.300
	192	0.379 $\pm$ 0.002	0.388	0.376 $\pm$ 0.001	0.380	0.541 $\pm$ 0.000	0.601	0.877	0.402	0.477	0.400	0.388	0.528	0.385
	336	0.419 $\pm$ 0.001	0.426	0.419 $\pm$ 0.002	0.428	0.608 $\pm$ 0.002	0.676	1.043	0.452	0.594	0.462	0.431	0.643	0.440
	720	0.424 $\pm$ 0.003	0.431	0.422 $\pm$ 0.000	0.427	0.608 $\pm$ 0.001	0.676	1.104	0.462	0.831	0.457	0.432	0.874	0.426
	Avg	0.379	0.387	0.377	0.383	0.545	0.605	0.942	0.414	0.559	0.410	0.390	0.611	0.388
ETT <h>1</h>	96	0.323 $\pm$ 0.001	0.329	0.321 $\pm$ 0.002	0.334	0.475 $\pm$ 0.003	0.528	0.404	0.338	0.345	0.342	0.349	0.364	0.344
	192	0.361 $\pm$ 0.000	0.367	0.369 $\pm$ 0.000	0.377	0.536 $\pm$ 0.001	0.596	0.450	0.374	0.380	0.385	0.385	0.398	0.384
	336	0.390 $\pm$ 0.002	0.399	0.413 $\pm$ 0.001	0.426	0.607 $\pm$ 0.000	0.674	0.532	0.410	0.413	0.402	0.412	0.428	0.442
	720	0.444 $\pm$ 0.000	0.454	0.470 $\pm$ 0.001	0.491	0.699 $\pm$ 0.002	0.777	0.666	0.478	0.474	0.472	0.474	0.487	0.479
	Avg	0.380	0.387	0.393	0.407	0.579	0.643	0.513	0.400	0.403	0.400	0.405	0.419	0.412
ETT <h>2</h>	96	0.168 $\pm$ 0.000	0.175	0.174 $\pm$ 0.001	0.180	0.257 $\pm$ 0.002	0.285	0.287	0.187	0.193	0.184	0.180	0.207	0.187
	192	0.233 $\pm$ 0.001	0.241	0.244 $\pm$ 0.002	0.250	0.356 $\pm$ 0.003	0.396	0.414	0.249	0.284	0.248	0.243	0.290	0.253
	336	0.296 $\pm$ 0.002	0.305	0.305 $\pm$ 0.003	0.311	0.443 $\pm$ 0.000	0.492	0.597	0.321	0.369	0.335	0.302	0.377	0.320
	720	0.390 $\pm$ 0.001	0.402	0.404 $\pm$ 0.000	0.412	0.587 $\pm$ 0.001	0.652	1.730	0.408	0.554	0.412	0.398	0.558	0.440
	Avg	0.272	0.281	0.282	0.288	0.410	0.456	0.757	0.291	0.350	0.295	0.281	0.358	0.300
ECL	96	0.186 $\pm$ 0.001	0.195	0.141 $\pm$ 0.000	0.148	0.234 $\pm$ 0.002	0.260	0.219	0.168	0.197	0.173	0.194	0.237	0.146
	192	0.192 $\pm$ 0.001	0.199	0.156 $\pm$ 0.002	0.162	0.239 $\pm$ 0.001	0.266	0.231	0.184	0.196	0.183	0.197	0.236	0.160
	336	0.203 $\pm$ 0.002	0.215	0.169 $\pm$ 0.001	0.178	0.252 $\pm$ 0.000	0.280	0.246	0.198	0.209	0.203	0.209	0.249	0.172
	720	0.245 $\pm$ 0.000	0.256	0.209 $\pm$ 0.003	0.225	0.272 $\pm$ 0.001	0.302	0.280	0.220	0.245	0.259	0.251	0.284	0.209
	Avg	0.207	0.216	0.169	0.178	0.249	0.277	0.244	0.192	0.212	0.205	0.213	0.251	0.172
Traffic	96	0.524 $\pm$ 0.002	0.544	0.381 $\pm$ 0.000	0.395	0.582 $\pm$ 0.001	0.647	0.522	0.593	0.650	0.529	0.543	0.805	0.403
	192	0.537 $\pm$ 0.001	0.540	0.412 $\pm$ 0.002	0.417	0.584 $\pm$ 0.004	0.649	0.530	0.617	0.598	0.533	0.555	0.756	0.416
	336	0.543 $\pm$ 0.000	0.551	0.429 $\pm$ 0.001	0.433	0.600 $\pm$ 0.002	0.667	0.558	0.629	0.605	0.545	0.565	0.762	0.448
	720	0.571 $\pm$ 0.001	0.586	0.462 $\pm$ 0.000	0.467	0.627 $\pm$ 0.001	0.697	0.589	0.640	0.645	0.583	0.639	0.719	0.471
	Avg	0.544	0.555	0.421	0.428	0.599	0.665	0.550	0.620	0.625	0.548	0.576	0.760	0.435
Weather	96	0.171 $\pm$ 0.000	0.177	0.165 $\pm$ 0.002	0.174	0.356 $\pm$ 0.002	0.395	0.158	0.172	0.196	0.162	0.180	0.202	0.171
	192	0.209 $\pm$ 0.002	0.225	0.210 $\pm$ 0.003	0.221	0.557 $\pm$ 0.001	0.619	0.206	0.219	0.237	0.211	0.226	0.242	0.219
	336	0.273 $\pm$ 0.001	0.278	0.266 $\pm$ 0.001	0.278	0.620 $\pm$ 0.002	0.689	0.272	0.280	0.283	0.267	0.281	0.287	0.286
	720	0.342 $\pm$ 0.001	0.354	0.350 $\pm$ 0.000	0.358	0.833 $\pm$ 0.000	0.926	0.398	0.365	0.345	0.343	0.358	0.351	0.357
	Avg	0.249	0.259	0.248	0.258	0.591	0.657	0.259	0.259	0.265	0.246	0.261	0.271	0.258
Solar-Energy	96	0.227 $\pm$ 0.000	0.234	0.197 $\pm$ 0.001	0.203	0.347 $\pm$ 0.002	0.386	0.310	0.250	0.290	0.247	0.245	0.312	0.204
	192	0.257 $\pm$ 0.001	0.267	0.227 $\pm$ 0.002	0.233	0.400 $\pm$ 0.001	0.444	0.734	0.296	0.320	0.288	0.255	0.339	0.249
	336	0.282 $\pm$ 0.002	0.290	0.242 $\pm$ 0.001	0.248	0.424 $\pm$ 0.001	0.471	0.750	0.319	0.353	0.329	0.259	0.368	0.260
	720	0.280 $\pm$ 0.004	0.289	0.243 $\pm$ 0.000	0.249	0.426 $\pm$ 0.000	0.473	0.769	0.338	0.356	0.341	0.262	0.370	0.271
	Avg	0.262	0.270	0.227	0.233	0.398	0.442	0.641	0.301	0.330	0.301	0.255	0.347	0.246

They are: Transformer [45], which uses temporal tokens; PatchTST [37], which uses variable tokens; and iTransformer [30], which also uses variable tokens. When applying TEM to the above methods, we use the same hyperparameters as reported in their original papers. Additional implementation details are provided in Appendix F. For the bi-level optimization, in the inner loop, we adopt the original update strategies and optimizers used by each of the aforementioned methods. In the outer loop, the positional/semantic topology weights are updated using the Adam optimizer with a decaying learning rate. The initial learning rate is set to $1e-3$.

5.2. Comparative Experimental Results

Table 1 and Table 2 present the comprehensive prediction results, where the best performance is highlighted in red and the second-best in blue. Lower MSE/MAE values indicate higher prediction accuracy. Compared to other time series forecasting (TSF) models, incorporating our method into various baseline models consistently improves performance. These experimental results demonstrate that our approach is applicable to different types of tokens, thereby validating the effectiveness of the proposed method.

5.3. Ablation Study

This subsection conducts ablation studies to systematically evaluate the contribution of the PTEM and STEM designs in the proposed method to model performance. We first analyze the performance changes after removing PTEM and STEM from the model to verify the effectiveness of each module. Next, we inject quantitative OPT and OST into each layer of the model and assess their impact on performance, thereby demonstrating the importance of

Table 2

Full MAE results for the TSF task. All baseline models' input length is 96, and the prediction lengths include {96, 192, 336, 720}. The results of our method are averaged over five random seeds, and the standard deviation across the five runs is reported after the “ $\pm$ ” symbol.

Models	PatchTST+TEM	PatchTST	iTransformer+TEM	iTransformer	Transformer+TEM	Transformer	Crossformer	TimesNet	DLinear	DSformer	SpareTSF	TIDE	PRReg	CI
Metric	(Ours) MAE	(2023) MAE	(Ours) MAE	(2024) MAE	(Ours) MAE	(2017) MAE	(2023) MAE	(2023) MAE	(2023) MAE	(2023) MAE	(2024) MAE	(2023) MAE	(2024) MAE	(2024) MAE
ETTh1	96	0.410 $\pm$ 0.000	0.419	0.400 $\pm$ 0.001	0.405	0.485 $\pm$ 0.001	0.539	0.448	0.402	0.400	0.412	0.390	0.464	0.408
	192	0.436 $\pm$ 0.001	0.445	0.431 $\pm$ 0.001	0.436	0.522 $\pm$ 0.001	0.580	0.474	0.429	0.432	0.435	0.420	0.492	0.447
	336	0.456 $\pm$ 0.002	0.466	0.450 $\pm$ 0.001	0.458	0.548 $\pm$ 0.002	0.609	0.546	0.469	0.459	0.456	0.440	0.515	0.483
	720	0.477 $\pm$ 0.003	0.488	0.482 $\pm$ 0.000	0.491	0.588 $\pm$ 0.001	0.653	0.621	0.500	0.516	0.475	0.472	0.558	0.490
	Avg	0.445	0.454	0.441	0.447	0.535	0.594	0.522	0.450	0.452	0.446	0.431	0.507	0.457
ETTm2	96	0.344 $\pm$ 0.001	0.348	0.344 $\pm$ 0.001	0.347	0.415 $\pm$ 0.001	0.461	0.584	0.374	0.387	0.359	0.354	0.440	0.363
	192	0.395 $\pm$ 0.002	0.400	0.397 $\pm$ 0.002	0.400	0.479 $\pm$ 0.001	0.532	0.656	0.414	0.476	0.410	0.398	0.509	0.414
	336	0.419 $\pm$ 0.001	0.433	0.427 $\pm$ 0.001	0.432	0.517 $\pm$ 0.002	0.575	0.731	0.452	0.541	0.451	0.443	0.571	0.452
	720	0.440 $\pm$ 0.003	0.446	0.435 $\pm$ 0.001	0.445	0.533 $\pm$ 0.000	0.592	0.763	0.468	0.657	0.463	0.447	0.679	0.466
	Avg	0.400	0.407	0.401	0.407	0.487	0.541	0.684	0.427	0.515	0.421	0.411	0.550	0.424
ETTm1	96	0.361 $\pm$ 0.001	0.367	0.359 $\pm$ 0.001	0.368	0.440 $\pm$ 0.001	0.489	0.426	0.375	0.372	0.375	0.381	0.387	0.384
	192	0.379 $\pm$ 0.001	0.385	0.387 $\pm$ 0.001	0.391	0.468 $\pm$ 0.001	0.520	0.451	0.387	0.389	0.392	0.397	0.404	0.399
	336	0.402 $\pm$ 0.002	0.410	0.410 $\pm$ 0.002	0.420	0.503 $\pm$ 0.002	0.559	0.515	0.411	0.413	0.411	0.417	0.425	0.426
	720	0.429 $\pm$ 0.002	0.439	0.447 $\pm$ 0.001	0.459	0.549 $\pm$ 0.003	0.610	0.589	0.450	0.453	0.448	0.450	0.461	0.455
	Avg	0.393	0.400	0.401	0.410	0.491	0.545	0.496	0.406	0.407	0.407	0.411	0.419	0.416
ETTm2	96	0.253 $\pm$ 0.001	0.259	0.258 $\pm$ 0.000	0.264	0.316 $\pm$ 0.001	0.351	0.366	0.267	0.292	0.269	0.259	0.305	0.278
	192	0.296 $\pm$ 0.000	0.302	0.303 $\pm$ 0.001	0.309	0.370 $\pm$ 0.002	0.411	0.492	0.309	0.362	0.306	0.308	0.364	0.329
	336	0.335 $\pm$ 0.002	0.343	0.341 $\pm$ 0.001	0.348	0.417 $\pm$ 0.002	0.463	0.542	0.351	0.427	0.365	0.342	0.422	0.366
	720	0.392 $\pm$ 0.003	0.400	0.400 $\pm$ 0.002	0.407	0.487 $\pm$ 0.003	0.541	1.042	0.403	0.522	0.409	0.398	0.524	0.428
	Avg	0.319	0.326	0.326	0.332	0.398	0.442	0.610	0.333	0.401	0.337	0.327	0.404	0.350
ECL	96	0.281 $\pm$ 0.001	0.285	0.235 $\pm$ 0.001	0.240	0.322 $\pm$ 0.001	0.358	0.314	0.272	0.282	0.269	0.269	0.329	0.239
	192	0.283 $\pm$ 0.001	0.289	0.252 $\pm$ 0.000	0.253	0.330 $\pm$ 0.000	0.367	0.322	0.289	0.285	0.280	0.273	0.330	0.262
	336	0.291 $\pm$ 0.002	0.305	0.264 $\pm$ 0.001	0.269	0.338 $\pm$ 0.002	0.375	0.337	0.300	0.301	0.297	0.287	0.344	0.276
	720	0.318 $\pm$ 0.003	0.337	0.305 $\pm$ 0.002	0.317	0.348 $\pm$ 0.002	0.386	0.363	0.320	0.333	0.340	0.321	0.373	0.306
	Avg	0.293	0.304	0.264	0.270	0.335	0.372	0.334	0.295	0.300	0.297	0.288	0.344	0.271
Traffic	96	0.351 $\pm$ 0.001	0.359	0.263 $\pm$ 0.001	0.268	0.321 $\pm$ 0.001	0.357	0.290	0.321	0.396	0.370	0.332	0.493	0.279
	192	0.353 $\pm$ 0.001	0.354	0.273 $\pm$ 0.001	0.276	0.320 $\pm$ 0.001	0.356	0.293	0.336	0.370	0.366	0.356	0.474	0.301
	336	0.355 $\pm$ 0.001	0.358	0.281 $\pm$ 0.002	0.283	0.328 $\pm$ 0.002	0.364	0.305	0.336	0.373	0.370	0.360	0.477	0.305
	720	0.361 $\pm$ 0.002	0.375	0.300 $\pm$ 0.001	0.302	0.338 $\pm$ 0.003	0.376	0.328	0.350	0.394	0.386	0.352	0.449	0.322
	Avg	0.355	0.362	0.279	0.282	0.327	0.363	0.304	0.336	0.383	0.373	0.350	0.473	0.302
Weather	96	0.208 $\pm$ 0.001	0.218	0.208 $\pm$ 0.000	0.214	0.384 $\pm$ 0.002	0.427	0.230	0.220	0.255	0.207	0.230	0.261	0.217
	192	0.255 $\pm$ 0.001	0.259	0.252 $\pm$ 0.001	0.254	0.504 $\pm$ 0.002	0.560	0.277	0.261	0.296	0.252	0.271	0.298	0.270
	336	0.294 $\pm$ 0.002	0.297	0.293 $\pm$ 0.002	0.296	0.535 $\pm$ 0.002	0.594	0.335	0.306	0.335	0.294	0.311	0.335	0.314
	720	0.340 $\pm$ 0.002	0.348	0.345 $\pm$ 0.001	0.349	0.639 $\pm$ 0.003	0.710	0.418	0.359	0.381	0.343	0.360	0.386	0.349
	Avg	0.274	0.281	0.275	0.279	0.515	0.572	0.315	0.287	0.317	0.274	0.293	0.320	0.288
Solar-Energy	96	0.280 $\pm$ 0.001	0.286	0.231 $\pm$ 0.001	0.237	0.335 $\pm$ 0.001	0.372	0.331	0.292	0.378	0.292	0.291	0.399	0.251
	192	0.302 $\pm$ 0.001	0.310	0.255 $\pm$ 0.001	0.261	0.368 $\pm$ 0.002	0.409	0.725	0.318	0.398	0.320	0.294	0.416	0.272
	336	0.304 $\pm$ 0.001	0.315	0.266 $\pm$ 0.002	0.273	0.386 $\pm$ 0.002	0.429	0.735	0.330	0.415	0.344	0.300	0.430	0.285
	720	0.309 $\pm$ 0.002	0.317	0.268 $\pm$ 0.000	0.275	0.389 $\pm$ 0.002	0.432	0.765	0.337	0.413	0.352	0.303	0.425	0.292
	Avg	0.299	0.307	0.255	0.262	0.370	0.411	0.639	0.319	0.401	0.327	0.297	0.417	0.275

the adaptive injection mechanisms in PTEM and STEM. Finally, we perform ablation studies on the adopted bi-level optimization strategy to verify its rationality and contribution.

Ablation Study of PTEM, STEM In this section, we conduct a systematic ablation study on the effects of the two proposed components, PTEM and STEM, across different models. Specifically, we progressively introduce STEM, PTEM, and their combination (referred to as TEM) into three representative models: PatchTST, iTransformer, and the vanilla Transformer. Experiments are performed on the ETTm1, Weather, and ECL datasets, which contain 7, 21, and 321 variables, respectively, representing three typical scenarios with small, medium, and large numbers of variables. The complete experimental results are shown in Tables 3, 4, and 5. The results demonstrate that incorporating PTEM and STEM consistently leads to stable and significant performance improvements across all three models. Taking MSE as an example, introducing STEM alone yields an average 2.32% reduction in error, while PTEM alone brings an average 4.48% reduction in error. When combining both modules (+TEM), the overall average MSE reduction reaches 5.81%, achieving the best performance compared to the baseline models. These findings indicate that PTEM and STEM each contribute independently and significantly to model performance, and their joint usage further enhances the model's ability to capture temporal structures and improve generalization.

Ablation Study of Adaptive Injection Mechanisms In Eq.(8) and (9), we introduce adaptive parameters Γ and Ξ to flexibly control the amount of OPT and OST injected into the features. To evaluate the importance of these adaptive mechanisms, we train and evaluate our model by setting Γ and Ξ to different fixed values. The experiments are conducted on three datasets: ETTm1, Weather, and ECL. The experiments are also conducted on three model

Table 3

Ablation study of PTEM and STEM on **PatchTST**. Look-back length is 96; prediction lengths $\in \{96, 192, 336, 720\}$. Lower is better; the best result in each row is highlighted in bold.

Dataset	Pred Len	PatchTST	+STEM	+PTEM	PatchTST +TEM (PTEM & STEM)
ETTM1	96	0.329 / 0.367	0.327 / 0.365	0.324 / 0.362	0.323 / 0.361
	192	0.367 / 0.385	0.365 / 0.383	0.362 / 0.380	0.361 / 0.379
	336	0.399 / 0.410	0.395 / 0.407	0.392 / 0.404	0.390 / 0.402
	720	0.454 / 0.439	0.450 / 0.435	0.446 / 0.431	0.444 / 0.429
	Avg	0.387 / 0.400	0.384 / 0.397	0.382 / 0.395	0.380 / 0.393
ECL	96	0.195 / 0.285	0.191 / 0.283	0.188 / 0.282	0.186 / 0.281
	192	0.199 / 0.289	0.196 / 0.287	0.193 / 0.285	0.192 / 0.283
	336	0.215 / 0.305	0.210 / 0.300	0.206 / 0.296	0.203 / 0.291
	720	0.256 / 0.337	0.252 / 0.329	0.247 / 0.323	0.245 / 0.318
	Avg	0.216 / 0.304	0.213 / 0.300	0.210 / 0.297	0.207 / 0.293
Weather	96	0.177 / 0.218	0.175 / 0.214	0.172 / 0.210	0.171 / 0.208
	192	0.225 / 0.259	0.219 / 0.257	0.212 / 0.256	0.209 / 0.255
	336	0.278 / 0.297	0.276 / 0.296	0.274 / 0.295	0.273 / 0.294
	720	0.354 / 0.348	0.349 / 0.345	0.346 / 0.342	0.342 / 0.340
	Avg	0.259 / 0.281	0.255 / 0.278	0.252 / 0.275	0.249 / 0.274

Table 4

Ablation study of PTEM and STEM on **iTransformer**. Look-back length is 96; prediction lengths $\in \{96, 192, 336, 720\}$. The best result in each row is highlighted in bold.

Dataset	Pred Len	iTransformer	+STEM	+PTEM	iTransformer +TEM (PTEM & STEM)
ETTM1	96	0.334 / 0.368	0.329 / 0.364	0.324 / 0.361	0.321 / 0.359
	192	0.377 / 0.391	0.374 / 0.390	0.371 / 0.389	0.369 / 0.387
	336	0.426 / 0.420	0.421 / 0.416	0.416 / 0.412	0.413 / 0.410
	720	0.491 / 0.459	0.483 / 0.454	0.474 / 0.449	0.470 / 0.447
	Avg	0.407 / 0.410	0.401 / 0.406	0.396 / 0.402	0.393 / 0.401
ECL	96	0.148 / 0.240	0.145 / 0.238	0.143 / 0.237	0.141 / 0.235
	192	0.162 / 0.253	0.160 / 0.253	0.158 / 0.252	0.156 / 0.252
	336	0.178 / 0.269	0.174 / 0.267	0.170 / 0.265	0.169 / 0.264
	720	0.225 / 0.317	0.219 / 0.312	0.213 / 0.307	0.209 / 0.305
	Avg	0.178 / 0.270	0.174 / 0.268	0.171 / 0.266	0.169 / 0.264
Weather	96	0.174 / 0.214	0.170 / 0.212	0.167 / 0.210	0.165 / 0.208
	192	0.221 / 0.254	0.217 / 0.253	0.212 / 0.252	0.210 / 0.252
	336	0.278 / 0.296	0.273 / 0.295	0.269 / 0.294	0.266 / 0.293
	720	0.358 / 0.349	0.355 / 0.347	0.352 / 0.346	0.350 / 0.345
	Avg	0.258 / 0.279	0.254 / 0.277	0.250 / 0.276	0.248 / 0.275

configurations: PatchTST+TEM, iTransformer+TEM, and Transformer+TEM. The experimental results are shown in Figure 4, 5, 6. The results show that injecting a fixed amount of OPT and OST into each layer of the network leads to unstable and inferior performance compared to the adaptive injection strategy. This demonstrates the effectiveness and rationality of the adaptive design in PTEM and STEM.

Ablation Study of Bi-level Optimization Strategy To ensure the accuracy of OPT/OST injection, we propose a bi-level optimization mechanism that jointly optimizes the TSF model (f_θ) and the parameters of PTEM and STEM modules (Γ, Ξ). To verify the rationality of this design, we replace the bi-level optimization with a regular joint optimization strategy, where the TSF model and the topology injection weights are updated simultaneously. We then train and evaluate the model on the ETTm1, Weather, and ECL datasets. In these experiments, both the input and prediction lengths are fixed at 96. The experiments are also conducted on three model configurations: PatchTST+TEM,

Table 5

Ablation study of PTEM and STEM on the vanilla **Transformer**. Look-back length is 96; prediction lengths $\in \{96, 192, 336, 720\}$. The best (lowest-error) result in each row is in bold.

Dataset	Pred Len	Transformer	+STEM	+PTEM	Transformer +TEM (PTEM & STEM)
ETTh1	96	0.528 / 0.489	0.507 / 0.469	0.486 / 0.450	0.475 / 0.440
	192	0.596 / 0.520	0.572 / 0.499	0.548 / 0.478	0.536 / 0.468
	336	0.674 / 0.559	0.647 / 0.537	0.620 / 0.514	0.607 / 0.503
	720	0.777 / 0.610	0.746 / 0.586	0.715 / 0.561	0.699 / 0.549
	Avg	0.643 / 0.545	0.617 / 0.523	0.592 / 0.502	0.579 / 0.491
ECL	96	0.260 / 0.358	0.250 / 0.344	0.240 / 0.330	0.234 / 0.322
	192	0.266 / 0.367	0.255 / 0.353	0.244 / 0.340	0.239 / 0.330
	336	0.280 / 0.375	0.269 / 0.360	0.259 / 0.348	0.252 / 0.338
	720	0.302 / 0.386	0.290 / 0.371	0.280 / 0.360	0.272 / 0.348
	Avg	0.277 / 0.372	0.266 / 0.357	0.255 / 0.344	0.249 / 0.335
Weather	96	0.395 / 0.427	0.379 / 0.410	0.363 / 0.394	0.356 / 0.384
	192	0.619 / 0.560	0.594 / 0.538	0.570 / 0.515	0.557 / 0.504
	336	0.689 / 0.594	0.662 / 0.571	0.635 / 0.547	0.620 / 0.535
	720	0.926 / 0.710	0.889 / 0.682	0.852 / 0.653	0.833 / 0.639
	Avg	0.657 / 0.572	0.631 / 0.549	0.605 / 0.537	0.591 / 0.515

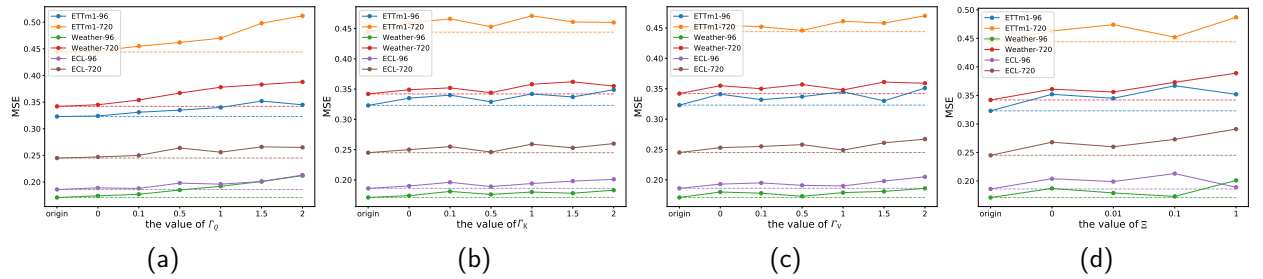


Figure 4: Illustration of our model's performance under different fixed values of $\Gamma_Q/\Gamma_K/\Gamma_V/\Xi$ across various datasets (ETTh1, Weather, and ECL) and prediction lengths (96 and 720). The algorithm adopts **PatchTST+TEM**. The first point on each curve represents the performance of using the adaptive weight. The dashed lines, drawn horizontally through the first points of each curve, are intended to facilitate comparison between the performance of using the adaptive weight and that under other fixed-value settings.

iTransformer+TEM, and Transformer+TEM. The mean squared errors (MSE) and training times across the datasets are shown in Figure 7. The experimental results show that the bi-level optimization strategy significantly outperforms the regular joint optimization strategy in terms of prediction performance, while incurring only a slight increase in training time (less than 20%). In particular, for model configurations with low training costs (e.g., ETTm1-PatchTST and ETTm1-iTransformer), bi-level optimization achieves notable performance improvements with almost no additional time overhead, as illustrated by the box icons in Figures 7a and 7b. These results further validate the rationality and effectiveness of the proposed design.

5.4. Hyperparameter Sensitivity

The only hyperparameter involved in this study is the initial learning rate of the outer loop (η_2). All other hyperparameters are inherited from the original settings of other baseline algorithms. In this section, we conduct a hyperparameter sensitivity analysis to determine the optimal value of the initial η_2 . The experiments are carried out on the ETTh1, Weather, and ECL datasets. As shown in Figure 8, the optimal value for the initial η_2 is found to be $1e-3$.

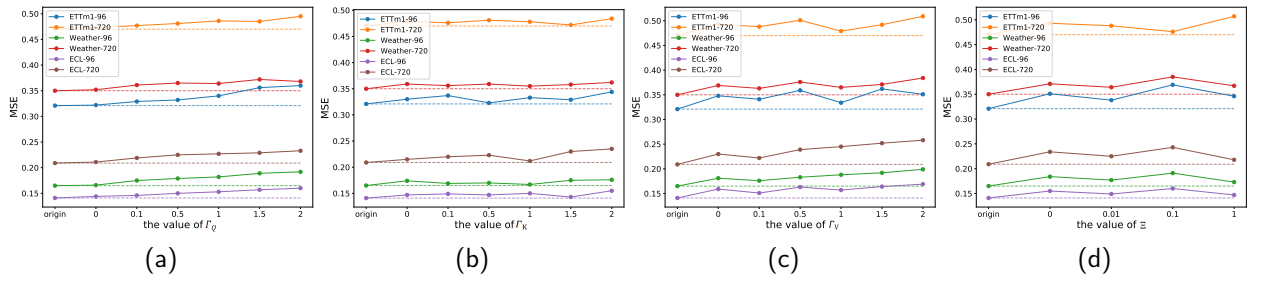


Figure 5: Illustration of our model's performance under different fixed values of $\Gamma_Q/\Gamma_K/\Gamma_V/\Xi$ across various datasets (ETTm1, Weather, and ECL) and prediction lengths (96 and 720). The algorithm adopts **iTransformer+TEM**. The setting is the same as Figure 4.

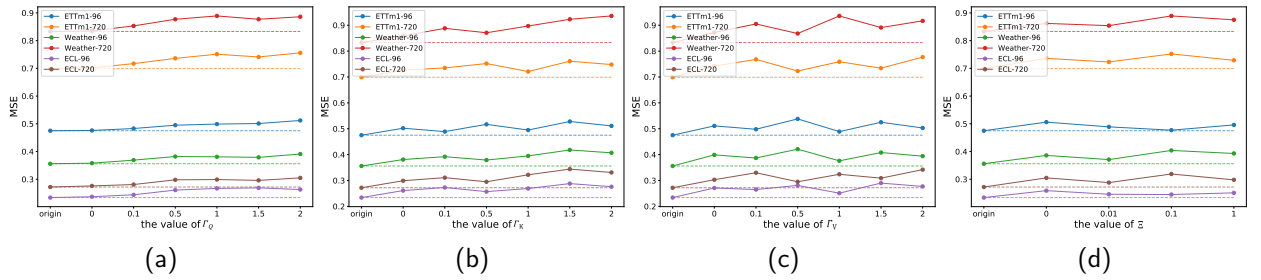


Figure 6: Illustration of our model's performance under different fixed values of $\Gamma_Q/\Gamma_K/\Gamma_V/\Xi$ across various datasets (ETTm1, Weather, and ECL) and prediction lengths (96 and 720). The algorithm adopts **Transformer+TEM**. The setting is the same as Figure 4.

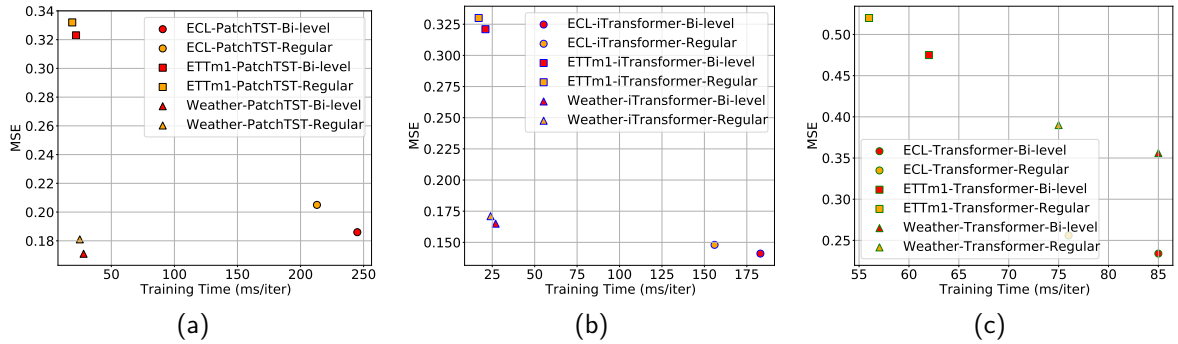


Figure 7: Comparison of training time and prediction accuracy between bi-level and regular optimization strategies. (a) PatchTST+TEM. (b) iTransformer+TEM. (c) Transformer+TEM.

5.5. Further Experiments Based on Section 3.2

To demonstrate the superiority of the adaptive PTEM and STEM approaches over the quantitative positional/semantic topology enhancement method used in Section 3.2, we integrate PTEM and STEM into the same three models (Transformer, PatchTST, iTransformer) used in that section. These models are then trained and evaluated under identical experimental settings. As illustrated in Figure 9 and Figure 10, employing either PTEM or STEM consistently yields significant performance gains across all datasets, with improvements exceeding those achieved by +EPT and +EST.

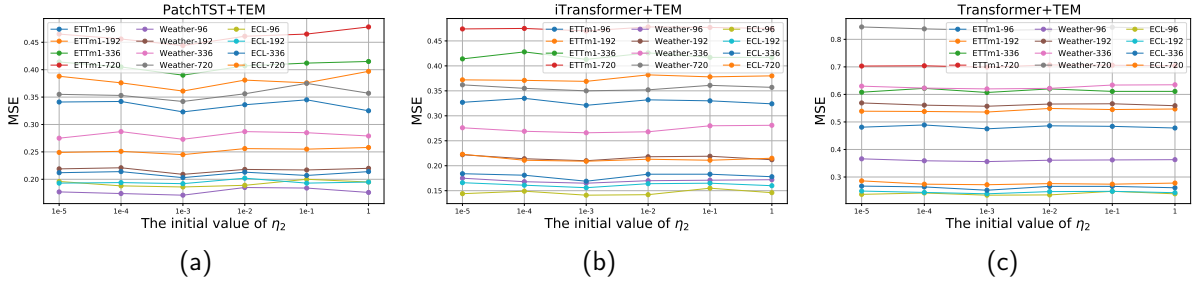


Figure 8: Hyperparameter sensitivity analysis of η_2^{init} . (a) PatchTST+TEM, (b) iTransformer+TEM, (c) Transformer+TEM.

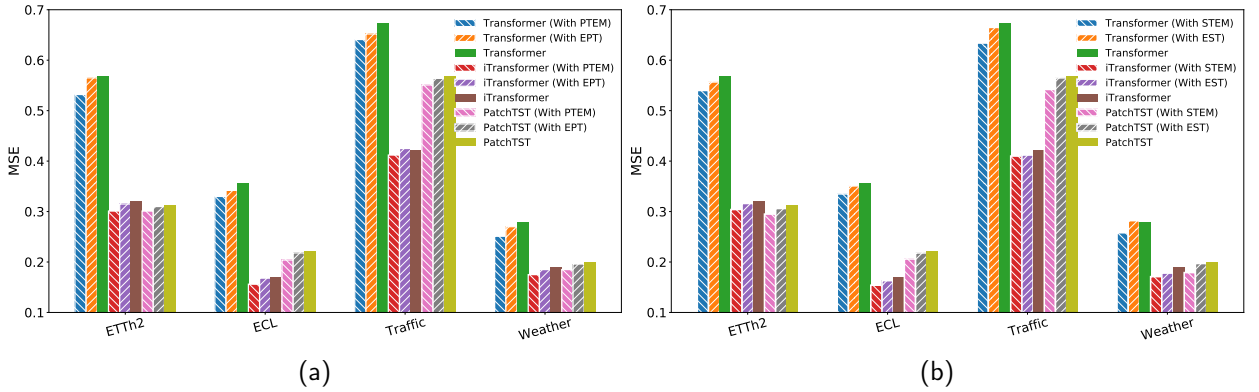


Figure 9: (a) Changes in model performance after applying PTEM to the models used in Figure 1c. (b) Changes in model performance after applying STEM to the models used in Figure 1d.

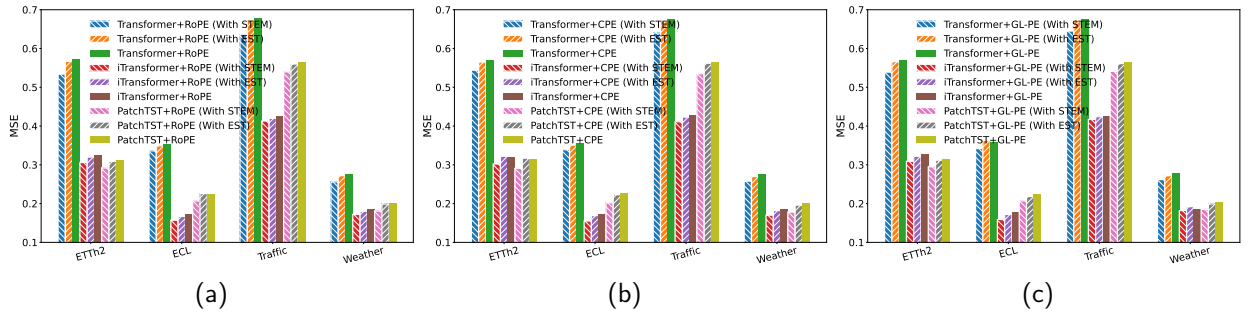
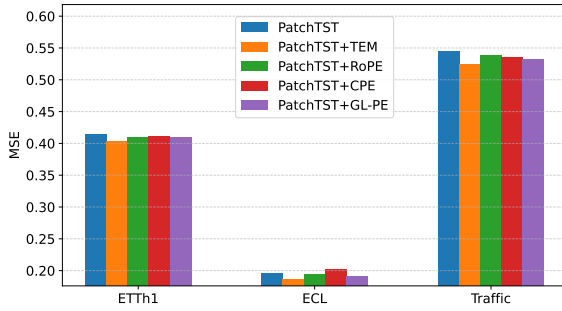


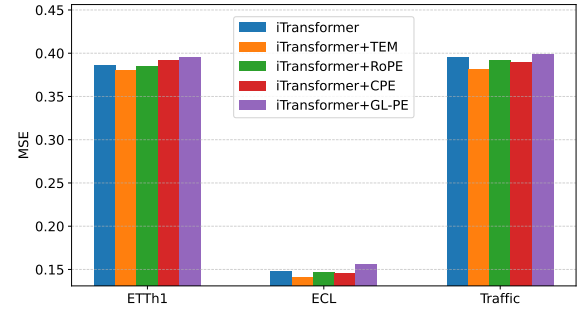
Figure 10: (a) Changes in model performance after applying STEM to the models used in Figure 2d. (b) Changes in model performance after applying STEM to the models used in Figure 2e. (c) Changes in model performance after applying STEM to the models used in Figure 2f.

5.6. Comparison with Other Positional Encodings (PEs)

Our approach is related to several studies on PE that likewise reinforce positional topology at deeper layers. For example, [43] introduces Rotary Position Embedding (RoPE) to improve Transformer performance on natural language processing tasks; [4] proposes Convolutional Positional Encoding (CPE) to enhance Transformers for vision tasks; and [32] presents GL-PE, which strengthens TSF by injecting fixed relative positional encodings at every layer. Note that these methods do not provide fine-grained control over the topological quantities injected into each layer, whereas our method does. In this subsection, we apply these three PEs and our proposed TEM to the same baseline method to

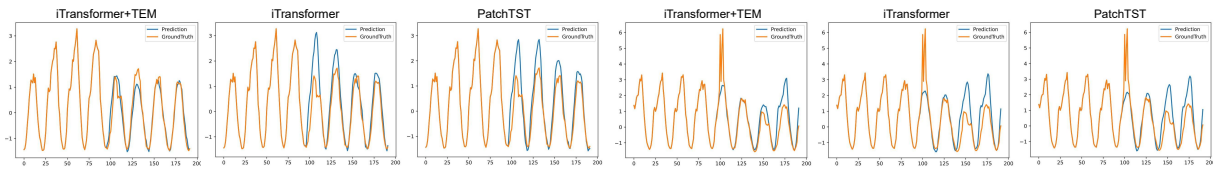


(a)

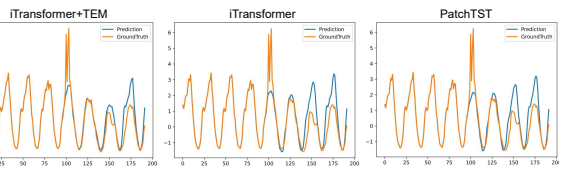


(b)

Figure 11: Comparison of the prediction performance of the proposed TEM and other positional encoding methods when applied to the TSF methods.

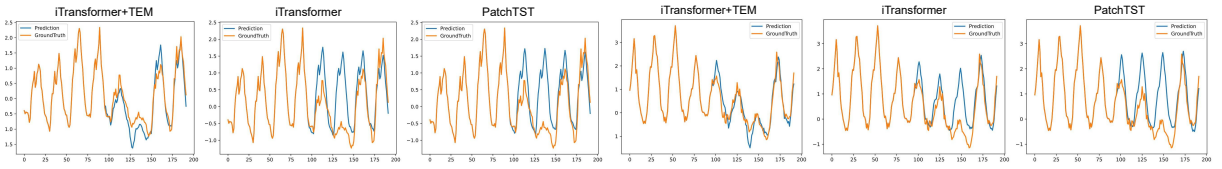


(a)

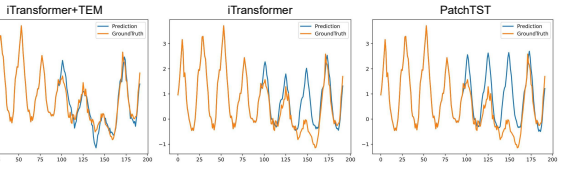


(b)

Figure 12: Figures (a) and (b) present two visualization examples of the Traffic dataset on the input-96-predict-96 task.

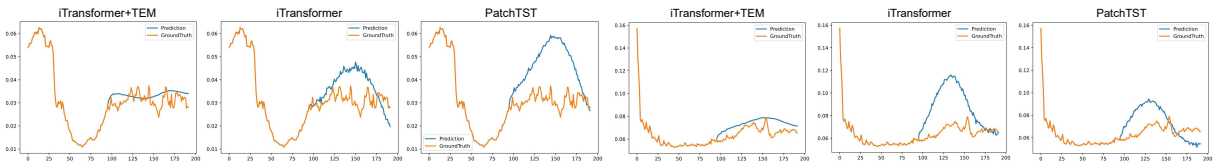


(a)

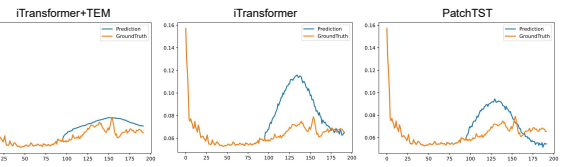


(b)

Figure 13: Figures (a) and (b) present two visualization examples of the ECL dataset on the input-96-predict-96 task.



(a)



(b)

Figure 14: Figures (a) and (b) present two visualization examples of the Weather dataset on the input-96-predict-96 task.

compare their effects. In all experiments, the look-back length and the prediction length are both fixed at 96. Results are shown in Figure 11. The experiments indicate that when these PEs are directly applied to Transformer-based TSF models, many methods yield limited gains and can even have negative effects in certain cases. We hypothesize that, due to the lack of fine-grained control over the amount of topology injected across layers, these three methods may introduce either too little or too much topological information, thereby weakening the model's predictive capability. In contrast, TEM employs an adaptive injection mechanism and introduces semantic topology enhancement, resulting in more stable and substantially larger improvements. These results further validate the effectiveness of our approach.

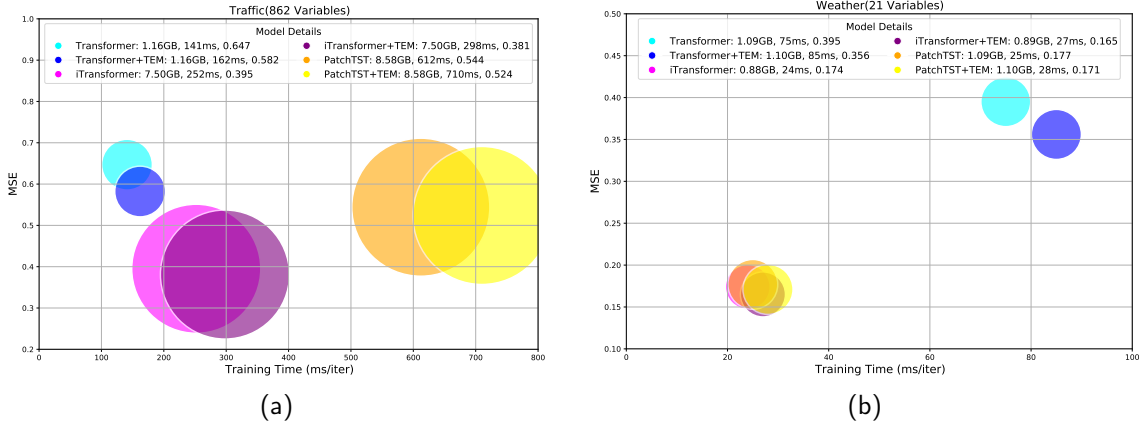


Figure 15: (a) Computational complexity analysis on the Traffic dataset. (b) Computational complexity analysis on the Weather dataset. Each item in the Model Details of the figure, from left to right, is: model name, GPU memory used, training speed, and forecasting MSE.

5.7. Computational Complexity Analysis

We conduct a comprehensive comparison of several models in terms of forecasting performance, training speed, and GPU memory usage. The experiments are conducted on the Weather and Traffic datasets, with both the input (look-back) length and prediction length set to 96. All models are implemented using their official configurations, with a batch size of 16. All experiments are performed on a single NVIDIA 4090 GPU with 24GB of GPU memory. The results are shown in Figures 15a and 15b. From the figures, we can observe the following: integrating our TEM into other Transformer-based methods introduces almost no increase in GPU memory usage, results in a slight increase in training time, and significantly improves forecasting performance.

5.8. Visualization of Prediction

To clearly compare the performance of different models, we present prediction visualizations for three representative datasets in Figures 12, 13, and 14. These results are generated using the following methods: our iTransformer+TEM, iTransformer [30], and PatchTST [37]. Among the various models, our method predicts future sequence variations with the highest accuracy and demonstrates superior performance.

6. Conclusion

This study empirically and theoretically reveals that as the depth of Transformer architectures increases, the original positional and semantic topological structures among input tokens tend to be progressively weakened, thereby limiting forecasting accuracy in time series tasks. To address this issue, we propose the Topology Enhancement Method (TEM), which introduces two key modules into Transformers: the Positional Topology Enhancement Module (PTEM) and the Semantic Topology Enhancement Module (STEM). These modules respectively preserve the original positional and semantic topology of input tokens. Both modules are equipped with learnable mechanisms and employ a bi-level optimization strategy to adaptively determine the optimal injection weights. As a plug-and-play framework, TEM can be seamlessly integrated into many mainstream Transformer-based models. Extensive experiments on multiple benchmark datasets demonstrate the superior performance of TEM.

CRedit authorship contribution statement

Jianqi Zhang: Conceptualization, Writing – original draft, Experimental analysis; **Wenwen Qiang:** Conceptualization, Funding acquisition, Supervision, Writing – review & editing; **Jingyao Wang:** Theoretical analysis, Writing – original draft, Writing – review & editing; **Jiahuan Zhou:** Writing – review & editing; **Changwen Zheng:** Funding acquisition, Supervision; **Hui Xiong:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

Our code and dataset are publicly available at: <https://github.com/jlu-phyComputer/TEM>.

Acknowledgements

This work is supported by the National Natural Science Foundation of China (No. 62506355).

References

- [1] Abhishek, K., Singh, M., Ghosh, S., Anand, A., 2012. Weather forecasting model using artificial neural network. *Procedia Technology* 4, 311–318.
- [2] Boussif, O., Boukachab, G., Assouline, D., Massaroli, S., Yuan, T., Benabbou, L., Bengio, Y., 2024. Improving* day-ahead* solar irradiance time series forecasting by leveraging spatio-temporal context. *Advances in Neural Information Processing Systems* 36.
- [3] Cai, D., He, X., Han, J., 2005. Document clustering using locality preserving indexing. *IEEE transactions on knowledge and data engineering* 17, 1624–1637.
- [4] Chu, X., Tian, Z., Zhang, B., Wang, X., Wei, X., Xia, H., Shen, C., 2021. Conditional positional encodings for vision transformers. *arXiv preprint arXiv:2102.10882*.
- [5] Cover, T.M., 1999. *Elements of information theory*. John Wiley & Sons.
- [6] Dai, R., Wang, Z., Jie, J., Wang, W., Ye, Q., 2025. Vtformer: a novel multiscale linear transformer forecaster with variate-temporal dependency for multivariate time series. *Complex & Intelligent Systems* 11, 1–19.
- [7] Das, A., Kong, W., Leach, A., Sen, R., Yu, R., 2023. Long-term forecasting with tide: Time-series dense encoder. *arXiv preprint arXiv:2304.08424*.
- [8] Devlin, J., Chang, M.W., Lee, K., Toutanova, K., 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- [9] Du, D., Su, B., Wei, Z., 2023. Preformer: predictive transformer with multi-scale segment-wise correlations for long-term time series forecasting, in: *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, IEEE. pp. 1–5.
- [10] Fang, Y., Qin, Y., Luo, H., Zhao, F., Zheng, K., 2023. Stwave+: A multi-scale efficient spectral graph attention network with long-term trends for disentangled traffic flow forecasting. *IEEE Transactions on Knowledge and Data Engineering*.
- [11] Gangeh, M.J., Zarkoob, H., Ghodsi, A., 2017. Fast and scalable feature selection for gene expression data using hilbert-schmidt independence criterion. *IEEE/ACM transactions on computational biology and bioinformatics* 14, 167–181.
- [12] Golowich, N., Rakhlin, A., Shamir, O., 2018. Size-independent sample complexity of neural networks, in: *Conference on Learning Theory (COLT)*, PMLR. pp. 297–299.
- [13] Greenfeld, D., Shalit, U., 2020. Robust learning with the hilbert-schmidt independence criterion, in: *International Conference on Machine Learning*, PMLR. pp. 3759–3768.
- [14] Han, L., Ye, H.J., Zhan, D.C., 2024. The capacity and robustness trade-off: Revisiting the channel independent strategy for multivariate time series forecasting. *IEEE Transactions on Knowledge and Data Engineering*.
- [15] He, X., Niyogi, P., 2003. Locality preserving projections. *Advances in neural information processing systems* 16.
- [16] He, X., Yan, S., Hu, Y., Niyogi, P., Zhang, H.J., 2005. Face recognition using laplacianfaces. *IEEE transactions on pattern analysis and machine intelligence* 27, 328–340.
- [17] Huang, J., Ma, M., Dai, Y., Hu, J., Du, S., 2023. Dbformer: A double-branch attention transformer for long-term time series forecasting. *Human-Centric Intelligent Systems* 3, 263–274.
- [18] Irani, H., Metsis, V., 2025a. Dywpe: Signal-aware dynamic wavelet positional encoding for time series transformers. *arXiv preprint arXiv:2509.14640*.
- [19] Irani, H., Metsis, V., 2025b. Positional encoding in transformer-based time series models: a survey. *arXiv preprint arXiv:2502.12370*.
- [20] Karevan, Z., Suykens, J.A., 2020. Transductive lstm for time-series prediction: An application to weather forecasting. *Neural Networks* 125, 1–9.
- [21] Ke, G., He, D., Liu, T.Y., 2020. Rethinking positional encoding in language pre-training. *arXiv preprint arXiv:2006.15595*.
- [22] Kingma, D.P., Ba, J., 2015. Adam: A method for stochastic optimization. *ICLR*.
- [23] Lai, G., Chang, W.C., Yang, Y., Liu, H., 2018. Modeling long-and short-term temporal patterns with deep neural networks. *SIGIR*.
- [24] Li, R., Zhang, F., Li, T., Zhang, N., Zhang, T., 2022. Dmgan: Dynamic multi-hop graph attention network for traffic forecasting. *IEEE Transactions on Knowledge and Data Engineering*.
- [25] Li, T., Yu, B., Li, J., Zhu, Z., 2024. Functional relation field: A model-agnostic framework for multivariate time series forecasting. *Artificial Intelligence* 334, 104158.
- [26] Liang, D., Zhang, H., Yuan, D., Ma, X., Li, D., Zhang, M., 2023. Does long-term series forecasting need complex attention and extra long inputs? *arXiv preprint arXiv:2306.05035*.

- [27] Lim, B., Arık, S.Ö., Loeff, N., Pfister, T., 2021. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting* 37, 1748–1764.
- [28] Lin, S., Lin, W., Wu, W., Chen, H., Yang, J., 2024. Sparsetsf: Modeling long-term time series forecasting with 1k parameters. *arXiv preprint arXiv:2405.00946*.
- [29] Liu, M., Zeng, A., Chen, M., Xu, Z., Lai, Q., Ma, L., Xu, Q., 2022. Scinet: Time series modeling and forecasting with sample convolution and interaction. *Advances in Neural Information Processing Systems* 35, 5816–5828.
- [30] Liu, Y., Hu, T., Zhang, H., Wu, H., Wang, S., Ma, L., Long, M., 2023. itransformer: Inverted transformers are effective for time series forecasting. *arXiv preprint arXiv:2310.06625*.
- [31] Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., Guo, B., 2021. Swin transformer: Hierarchical vision transformer using shifted windows, in: *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 10012–10022.
- [32] Lv, C., Wang, Y., Han, D., Shen, Y., Zheng, X., Huang, X., Li, D., 2025. Toward relative positional encoding in spiking transformers. *arXiv preprint arXiv:2501.16745*.
- [33] Messou, F.J.A., Chen, J., Liu, T., Zhang, S., Yu, K., 2025. Tsformer: Temporal-aware transformer for multi-horizon forecasting with learnable positional encodings and attention mechanisms, in: *2025 Sixteenth International Conference on Ubiquitous and Future Networks (ICUFN)*, IEEE. pp. 600–605.
- [34] Mohri, M., 2018. Foundations of machine learning.
- [35] Mohri, M., Rostamizadeh, A., 2008. Rademacher complexity bounds for non-iid processes. *Advances in neural information processing systems* 21.
- [36] Neyshabur, B., Bhojanapalli, S., McAllester, D., Srebro, N., 2018. The role of over-parametrization in generalization of neural networks, in: *International Conference on Learning Representations (ICLR)*.
- [37] Nie, Y., Nguyen, N.H., Sinthong, P., Kalagnanam, J., 2023. A time series is worth 64 words: Long-term forecasting with transformers. *ICLR*.
- [38] Novo, R., Marocco, P., Giorgi, G., Lanzini, A., Santarelli, M., Mattiazzo, G., 2022. Planning the decarbonisation of energy systems: The importance of applying time series clustering to long-term models. *Energy Conversion and Management: X* 15, 100274.
- [39] Pérez-Suay, A., Camps-Valls, G., 2018. Sensitivity maps of the hilbert–schmidt independence criterion. *Applied Soft Computing* 70, 1054–1063.
- [40] Qiang, W., Li, J., Zheng, C., Su, B., Xiong, H., 2021. Robust local preserving and global aligning network for adversarial domain adaptation. *IEEE Transactions on Knowledge and Data Engineering* 35, 3014–3029.
- [41] Sbrana, A., Rossi, A.L.D., Naldi, M.C., 2020. N-beats-rnn: deep learning for time series forecasting, in: *2020 19th IEEE International Conference on Machine Learning and Applications (ICMLA)*, IEEE. pp. 765–768.
- [42] Shaw, P., Uszkoreit, J., Vaswani, A., 2018. Self-attention with relative position representations. *arXiv preprint arXiv:1803.02155*.
- [43] Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., Liu, Y., 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing* 568, 127063.
- [44] Tang, P., Zhang, X., 2023. Infomaxformer: Maximum entropy transformer for long time-series forecasting problem. *arXiv preprint arXiv:2301.01772*.
- [45] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I., 2017. Attention is all you need. *NeurIPS*.
- [46] Wang, H., Chen, J., Fan, Z., Zhang, Z., Cai, Z., Song, X., 2022. St-expertnet: A deep expert framework for traffic prediction. *IEEE Transactions on Knowledge and Data Engineering*.
- [47] Wang, T., Dai, X., Liu, Y., 2021. Learning with hilbert–schmidt independence criterion: A review and new perspectives. *Knowledge-based systems* 234, 107567.
- [48] Wang, Y., Liu, C., Yuan, X., 2020. Stacked locality preserving autoencoder for feature extraction and its application for industrial process data modeling. *Chemometrics and Intelligent Laboratory Systems* 203, 104086.
- [49] Wu, H., Hu, T., Liu, Y., Zhou, H., Wang, J., Long, M., 2023. Timesnet: Temporal 2d-variation modeling for general time series analysis. *ICLR*.
- [50] Wu, H., Xu, J., Wang, J., Long, M., 2021. Autoformer: Decomposition transformers with Auto-Correlation for long-term series forecasting. *NeurIPS*.
- [51] Wu, N., Green, B., Ben, X., O’Banion, S., 2020. Deep transformer models for time series forecasting: The influenza prevalence case. *arXiv preprint arXiv:2001.08317*.
- [52] Yu, C., Wang, F., Shao, Z., Sun, T., Wu, L., Xu, Y., 2023. Dsformer: a double sampling transformer for multivariate time series long-term prediction, in: *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, pp. 3062–3072.
- [53] Zeng, A., Chen, M., Zhang, L., Xu, Q., 2023. Are transformers effective for time series forecasting? *AAAI*.
- [54] Zhang, H., Qiang, W., Zhang, J., Chen, Y., Jing, L., 2022a. Unified feature extraction framework based on contrastive learning. *Knowledge-Based Systems* 258, 110028.
- [55] Zhang, H., Zhao, S., Qiang, W., Chen, Y., Jing, L., 2022b. Feature extraction framework based on contrastive learning with adaptive positive and negative samples. *Neural Networks* 156, 244–257.
- [56] Zhang, Y., Yan, J., 2023. Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting. *ICLR*.
- [57] Zhao, Y., Ma, Z., Zhou, T., Ye, M., Sun, L., Qian, Y., 2023. Gcformer: An efficient solution for accurate and scalable long-term multivariate time series forecasting, in: *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, pp. 3464–3473.
- [58] Zhou, H., Li, J., Zhang, S., Zhang, S., Yan, M., Xiong, H., 2023. Expanding the prediction capacity in long sequence time-series forecasting. *Artificial Intelligence* 318, 103886.
- [59] Zhou, T., Ma, Z., Wen, Q., Wang, X., Sun, L., Jin, R., 2022. FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting. *ICML*.

A. Appendix Overview

This supplementary material provides results for additional experiments and details to reproduce our results that could not be included in the paper submission due to space limitations.

- **Appendix B** provides the Table which summary all definitions of the notations from the main text.
- **Appendix C** provides three different tokenization strategy.
- **Appendix D** explains why adding the similarity matrix of input tokens to the $QK^\top$ can enhance the model's ability to preserve OST.
- **Appendix E** describes the dataset used in our paper.
- **Appendix F** provides more details about our method.
- **Appendix G** provides the pseudo-code.
- **Appendix H** provides proofs and further theoretical analysis of the theory in the text.
- **Appendix I** provides more details about our Experiments.
- **Appendix J** describes how TEM is integrated into our Concise Dual-branch Transformer Framework (CDTF) and outlines the network architecture and implementation. Extensive multi-dataset comparisons and ablation studies further demonstrate TEM's marked effectiveness and broad applicability within the dual-branch model.

Note that before we illustrate the details and analysis, we provide a brief summary about all the experiments conducted in this paper, as shown in Table 6.

B. Table of Notations

We list the definitions of all notations in Table 7 and 8.

C. Token Acquisition

The mainstream Transformer-based TSF methods primarily use three types of tokens: 1) Temporal tokens, which include all variables at the same timestamp; 2) Variable tokens, which contain all input time points of a specific variable; 3) Patch tokens, which consist of single variables from adjacent temporal regions. Temporal tokens are primarily used by earlier methods such as Transformer [45], Informer [58]. iTransformer [30] first proposes variable tokens. PatchTST [37] first proposes patch tokens. We provide a visual representation of the generation processes for these three types of tokens in Figure 16 to help readers better understand them. Let the input series be $X \in \mathbb{R}^{T \times N}$, where T is the number of time points and N is the number of variables. Then, X can be converted into T temporal tokens or N variable tokens. If the length of each patch is l , X can be converted into $N * T/l$ patch tokens.

D. Why Adding Similarity Matrix to $QK^\top$ Enhances Preservation of OST

This section explains why adding the similarity matrix of input tokens to the $QK^\top$ can enhance the model's ability to preserve OST.

First, the model's ability to preserve OST can be represented by the dependency between the similarity matrix of the input tokens and that of the output tokens at each layer. A stronger dependency indicates a better preservation ability. In the self-attention mechanism, the new token is computed as follows:

$$V^{new} = \text{softmax}\left(\frac{QK^\top}{\sqrt{D}}\right)V, \quad (13)$$

Table 6

Illustration of the experiments conducted in this work. Note that all experimental results are obtained after five rounds of experiments.

Experiments	Location	Results
Comparative experimental results	Section 5.2 and Appendix I.3	Table 1, Table 2 and Table 10
Ablation Study about PTEM, STEM	Section 5.3	Tables 3, Tables4, and Tables 5
Ablation Study about Adaptive Injection Mechanisms	Section 5.3	Figure 4, Figure 5 and Figure 6
Ablation Study of Bi-level Optimization Strategy	Section 5.3	Figure 7a and Figure 7b
Hyperparameter sensitivity	Section 5.4	Figure 8
Computational complexity analysis	Section 5.7	Figure 15a and Figure 15b
Visualization of Prediction	Section 5.8	Figure 12, Figure 13, and Figure 14
The further experiments based on Section 3.2	Section 5.5	Figure 9 and Figure 10
Comparison with Other Positional Encodings (PEs)	Section 5.6	Figure 11
The Importance of Position Encoding for Variable Tokens	Appendix I.4	Table 11
Comparative experimental results of our Concise Dual-branch Transformer Framework with TEM	Appendix J.3	Table 13 and Table 14
Ablation study of our Concise Dual-branch Transformer Framework with TEM	Appendix J.3	Table 15

where $Q, K, V \in \mathbb{R}^{N_t \times D}$ are the Query, Key, and Value matrix, respectively. $V^{new} \in \mathbb{R}^{N_t \times D}$ is the new token series generated by the attention layer. Let $\varpi \in \mathbb{R}^{N_t \times N_t}$ be $\text{softmax}(\frac{QK^T}{\sqrt{D}})$. We further transform Eq.(13) as:

$$V_i^{new} = \sum_{j=1}^{N_t} \varpi_{i,j} V_j, \quad i = 1, \dots, N_t, \quad (14)$$

where $V_i \in \mathbb{R}^{1 \times D}$ ($i = 1, \dots, N_t$) is the i -th token of V . $\varpi_{i,j}$ is the attention score between V_i and V_j . Let the input token series be $H^0 \in \mathbb{R}^{N_t \times D_{init}}$. D_{init} is the dimension of the input tokens. $H_i^0 \in \mathbb{R}^{1 \times D_{init}}$ ($i = 1, \dots, N_t$) is the i -th token of H^0 . The similarity matrix is calculated by:

$$\varpi^{init} = H^0 [H^0]^T \quad (15)$$

In the main text, we enhance the OST by adding ϖ^{init} into QK^T . If the similarity of $H_{k_1}^0$ and $H_{k_2}^0$ is relatively high, the value of ϖ_{k_1, k_2}^{init} is relatively high. In this case, when we add ϖ^{init} into QK^T , the ϖ_{k_1, k_2} is increased. According to Eq.(14), the generated $V_{k_1}^{new}$ will contain more information from $V_{k_2}^{new}$, thereby increasing their similarity. Therefore, adding the similarity matrix of the initial tokens to QV^T will result in initially similar tokens having higher similarity

Table 7
The definitions of notations (Part I).

Notations	Definition
Notations of Data	
$X = \{\mathbf{x}_1, \dots, \mathbf{x}_T\} \in \mathbb{R}^{T \times N}$	The input series
$Y = \{\mathbf{y}_{T+1}, \dots, \mathbf{y}_{T+S}\} \in \mathbb{R}^{S \times N}$	The output series
$\mathbf{x}_i$	The series data of the i -th time point
T	The look-back length
N	The number of variables in the input/output series
S	The prediction length
Notations of Topological Relationships	
$\mathcal{G}$	Positional Topology
$\mathcal{P} = \{1, \dots, N_t\}$	The index set of tokens' positions
N_t	The number of the token in the token series
Υ	The class of functions used to generate positional encodings (PE)
p	The functions used to generate positional encodings (PE)
$p(k)$	The positional encoding (PE) vector of the k -th position
H_i^l	the i -th intermediate-layer token feature
H^0	the input tokens
S^l	The <i>semantic topology</i> at the l -th layer
$\text{sim}(\cdot)$	The similarity function
Θ	The original local structure among input tokens
OPT	Original Positional Topology
OST, S^0	Original Semantic Topology

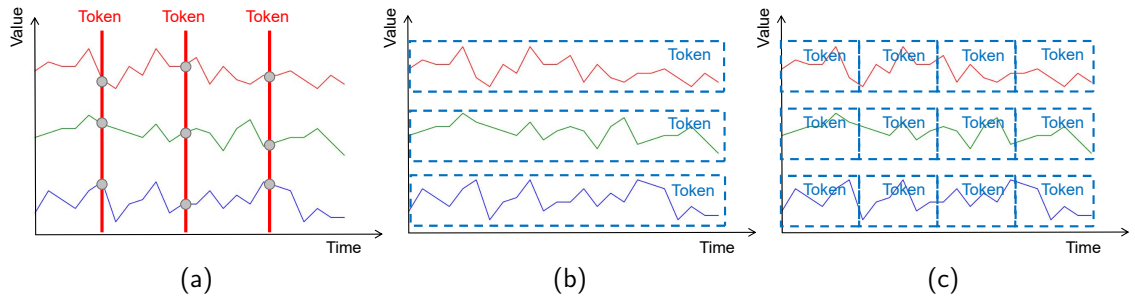


Figure 16: Visualization of the generation of two types of tokens. (a) temporal tokens, (b) variable tokens, (c) patch token.

after being processed by the Transformer layer. As a result, the dependency between the similarity matrix of the new tokens and that of the input tokens becomes stronger, which enhances the model's ability to preserve OST.

E. Dataset Descriptions

In this paper, we performed tests using twelve real-world datasets. These include: 1) ETT [58], which encompasses seven variables related to electricity transformers, gathered from July 2016 to July 2018. There are four subsets where ETTh1 and ETTh2 are recorded every hour, and ETTm1 and ETTm2 are recorded every 15 minutes. 2) Weather [50], covering 21 meteorological variables, was recorded at 10-minute intervals in 2020 by the Max Planck Institute for Biogeochemistry's Weather Station. 3) ECL [50], detailing the hourly electrical usage of 321 customers. 4) Traffic [50], which compiles data on the hourly occupancy rates of roads, monitored by 862 sensors in the San Francisco Bay area's freeways, spanning from January 2015 to December 2016. 5) Solar-Energy [23], documenting the solar energy output from 137 photovoltaic (PV) plants in 2006, with data points every 10 minutes. 6) PEMS contains the public traffic network data in California collected by 5-minute windows. We use the same four public subsets (PEMS03, PEMS04, PEMS07, PEMS08) adopted in SCINet [29].

We follow the same data processing and train-validation-test set split protocol used in iTransformer [30], where the train, validation, and test datasets are strictly divided according to chronological order to make sure there are no data leakage issues. As for the forecasting settings, the lookback length for datasets ETT, Weather, ECL, Solar-Energy, and

Table 8

The definitions of notations (Part II).

Notations	Definition
Notations of Model	
f_θ	The TSF (time series forecasting) model with parameters θ
Γ	The learnable coefficients for injecting OPT into each Transformer layer
$\Gamma_j^{l,i}$	The injection strength of OPT into path j (Query, Key, or Value) in head i of layer l
$\Gamma[l, i, j]$	Tensor notation representing $\Gamma_j^{l,i}$
Ξ	The learnable coefficients for injecting OST into each attention head
$\Xi_{l,i}$	The injection strength of OST into the i -th head of the l -th layer
$\Xi[l, i]$	Tensor notation representing $\Xi_{l,i}$
H^l	The input to the $(l+1)$ -th Transformer layer
$H_j^{l,i}$	The modified input for path j in head i of layer $l+1$
$QK^\top$	The dot-product similarity matrix between Query and Key
$\mathcal{L}_{\text{mse}}$	The mean squared error loss used in TSF
$\mathcal{D}_b = \{X_i, Y_i\}_{i=1}^{N_b}$	A mini-batch of N_b samples
f_θ^1	The updated model parameters after inner-loop optimization
η_1	The learning rate for the inner loop
η_2	The learning rate for the outer loop
$\nabla_{f_\theta} \mathcal{L}_{\text{mse}}$	Gradient of the loss with respect to model parameters
$\nabla_{\{\Gamma, \Xi\}} \mathcal{L}_{\text{mse}}$	Gradient of the loss with respect to topology injection parameters
Notations of Theory	
$\mathcal{D} = \{(X_i, Y_i)\}_{i=1}^m$	The training set
$\beta(\cdot)$	The mixing coefficient of the β -mixing distribution
μ	The number of mutually independent segments
b	The length of each mutually independent segment
$\mathcal{X}$	The input spaces for TSF
$\mathcal{Y}$	The output spaces for TSF
$\mathcal{H}$	The function class of Transformer-based functions
$h : \mathcal{X} \rightarrow \mathcal{Y}$	The function of Transformer-based functions
$\mathcal{H}_s^i$	A function class of the i -th Transformer layer
$\ell : \mathcal{Y} \times \mathcal{Y}$	A non-negative, ξ -Lipschitz loss function
M	The up bounded of ℓ
δ	The confidence level
$\mathfrak{R}_\mu(\cdot)$	The Rademacher complexity
$G(\cdot)$	An aggregation function over the layer-wise complexities
$p_i(k)$	The position encoding of the k -th token perceived from OPT at layer i
$H_k^i \in \mathbb{R}^{d_i}$	The representation of the k -th token of the output tokens of layer i
d_i	The dimension of tokens of layer i
$\Delta_{\mathcal{G}}^i$	The positional topology distortion at layer i
Δ_S^i	The semantic topology distortion at layer i
$\mathfrak{R}(\mathcal{H}_s^l)$	The Rademacher complexity of the l -th Transformer layer
$\mathfrak{R}(\mathcal{H}_s^0)$	The Rademacher complexity of the input layer
L_i	The Lipschitz constant of the i -th Transformer layer
ρ_i	The topology preservation ratio at layer i
α	The weight for positional topology distortion
β	The weight for semantic topology distortion

Traffic is set to 96, while their prediction length varies in $\{96, 192, 336, 720\}$. For PEMS, the lookback length is set to 96, and their prediction length varies in $\{12, 24, 36, 48\}$ which is the same as SCINet [29]. The details of the datasets are provided in Table 9.

F. Method Details

In this section, we present the methodological details. First, we describe the details of the Positional Topology Enhancement Module (PTEM) and the Semantic Topology Enhancement Module (STEM). Then, we provide the specific details of how our method is applied to Transformer [45], iTransformer [30], and PatchTST [37].

F.1. More Details of Positional Topology Enhancement Module (PTEM)

According to Eq.(8), PTEM adds the weighted OPT to the input of each Transformer layer. As Definition 3 and Definition 3.1, OPT essentially represents the positional information of tokens. Since the Transformer models token positions using the positional embedding (PE), OPT is essentially a form of PE. In time series forecasting (TSF), common positional encodings include sinusoidal PE or convolutional PE. For temporal tokens and patch tokens, this

Table 9

Detailed dataset descriptions. **Dim** denotes the variate number of each dataset. **Dataset Size** represents the number of time points in the (Train, Validation, Test) splits, respectively. **Prediction Length** denotes the future time points to be predicted, with four settings included per dataset. **Frequency** indicates the sampling interval of time points.

Dataset	Dim	Prediction Length	Dataset Size	Frequency	Information
ETTh1, ETTh2	7	{96, 192, 336, 720}	(8545, 2881, 2881)	Hourly	Electricity
ETTm1, ETTm2	7	{96, 192, 336, 720}	(34465, 11521, 11521)	15min	Electricity
Weather	21	{96, 192, 336, 720}	(36792, 5271, 10540)	10min	Weather
ECL	321	{96, 192, 336, 720}	(18317, 2633, 5261)	Hourly	Electricity
Traffic	862	{96, 192, 336, 720}	(12185, 1757, 3509)	Hourly	Transportation
Solar-Energy	137	{96, 192, 336, 720}	(36601, 5161, 10417)	10min	Energy
PEMS03	358	{12, 24, 48, 96}	(15617, 5135, 5135)	5min	Transportation
PEMS04	307	{12, 24, 48, 96}	(10172, 3375, 281)	5min	Transportation
PEMS07	883	{12, 24, 48, 96}	(16911, 5622, 468)	5min	Transportation
PEMS08	170	{12, 24, 48, 96}	(10690, 3548, 265)	5min	Transportation

paper adopts sinusoidal PE. For variable tokens, each token represents a specific input variable, and permuting the positions of tokens in the variable token sequence does not alter the semantic meaning of the input—i.e., it exhibits permutation invariance. Therefore, convolutional PE, which also possesses permutation invariance, is more suitable for variable tokens.

F.2. More Details of Semantic Topology Enhancement Module (STEM)

According to Eq.(9), STEM adds the weighted OST to QK^T of each Transformer layer. As Definition 3 and Definition 3.2, OST essentially represents the similarity of input tokens. We use a similarity matrix of input tokens whose shape is the same as QK^T to represent OST. The similarity matrix is calculated by:

$$\varpi^{init} = H^0[H^0]^T, \quad (16)$$

where H^0 is the input tokens.

F.3. Applying PTEM and STEM to Transformer

Transformer [45] uses temporal tokens to process time series forecasting. It employs multiple Transformer layers as the encoder and multiple Transformer layers with cross-attention mechanisms as the decoder. The encoder structure is consistent with the encoder shown in Figure 3(a). When applying PTEM and STEM to the Transformer model, we integrate them only into the encoder. The method of applying PTEM and STEM to the Transformer encoder can be referred to in Figure 3(b). The PE used is the sinusoidal PE.

F.4. Applying PTEM and STEM to iTransformer

iTransformer [30] uses variable tokens to handle time series forecasting tasks. It employs multiple Transformer layers as the encoder and uses a linear layer as the decoder. The encoder structure is consistent with the one shown in Figure 3(a). When applying PTEM and STEM to the iTransformer model, we integrate them only into the encoder. The method of applying PTEM and STEM to the iTransformer encoder can be referred to in Figure 3(b). The PE used is the convolutional PE.

F.5. Applying PTEM and STEM to PatchTST

PatchTST [37] uses patch tokens to handle time series forecasting tasks. It also employs multiple Transformer layers as the encoder and uses a linear layer as the decoder. The encoder structure is consistent with the one shown in Figure 3(a). When applying PTEM and STEM to the PatchTST model, we integrate them only into the encoder. The method of applying PTEM and STEM to the PatchTST encoder can be referred to in Figure 3(b). The PE used is the sinusoidal PE.

G. Pseudo-code

To facilitate the reader's understanding of the proposed algorithm, we present the pseudo-code of our method (Algorithm 1). The pseudo-code is illustrated using a single-branch method as an example.

Algorithm 1 TEM - Overall Architecture.

Require: Input time series $X \in \mathbb{R}^{T \times N}$; input Length T ; predicted length S ; variates number N ; token dimension D ; the number of attention layers L ; the number of head n_h ; the tokenization strategy f_{token} ; the embedding layer f_{Embed} ; the layer normalization (LN); the Feed-Forward Network (FFN); the linear layer $\mathbf{FC}_q$ to generate Q ; the linear layer $\mathbf{FC}_k$ to generate K ; the linear layer $\mathbf{FC}_v$ to generate V .

```
1:  $H^0 = f_{token}(X)$ 
2:  $OST = H^0 [H^0]^\top$ 
3:  $H^0 = f_{Embed}(H^0) + PE$ 
4:  $OPT = PE$ 
5: for  $l = 0$  to  $L - 1$  do
6:   Split  $H^l$  to  $H^{l,i} (i = 1, \dots, n_l)$ 
7:   for  $i = 1$  to  $n_h$  do
8:      $Q = \mathbf{FC}_q(H^{l,i} + \Gamma_Q^{l,i} \times OPT)$ 
9:      $K = \mathbf{FC}_k(H^{l,i} + \Gamma_K^{l,i} \times OPT)$ 
10:     $V = \mathbf{FC}_v(H^{l,i} + \Gamma_V^{l,i} \times OPT)$ 
11:     $A_{init} = QK^\top + \Xi_{l,i} \times OST$ 
12:     $H_{new}^{l,i} = softmax(\frac{A_{init}}{\sqrt{D}})V$ 
13:     $H^{l,i} = LN(H^{l,i} + H_{new}^{l,i})$ 
14:   end for
15:   Fuse  $H^{l,i} (i = 1, \dots, n_l)$  to  $H^l$ 
16:    $H^{l+1} = LN(H^l + FFN(H^l))$ 
17: end for
18:  $\hat{Y} = \text{Decoder}(H_L)$ 
19: return  $\hat{Y}$ 
```

H. Theoretical Analysis

H.1. Proof of Theorem 3.4

Proof. We begin by invoking the generalization bound for stationary β -mixing sequences (Theorem 2 in [35]). Let

$$\mathcal{F} := \{(x, y) \mapsto \ell(h(x), y) \mid h \in \mathcal{H}\}$$

be the composite loss function class. Assume the training dataset $\mathcal{D} = \{(X_i, Y_i)\}_{i=1}^m$ is generated from a stationary β -mixing process with coefficient $\beta(\cdot)$.

Let $m = 2\mu a$ for integers μ and a . Partition the dataset into 2μ blocks of size a , and define

$$\tilde{\mathcal{D}} := \{(\tilde{X}_i, \tilde{Y}_i)\}_{i=1}^\mu$$

by selecting every other block to approximate independence. Then, with probability at least $1 - \delta$, we have

$$\mathbb{E}_{(X,Y)}[\ell(h(X), Y)] \leq \frac{1}{\mu} \sum_{i=1}^\mu \ell(h(\tilde{X}_i), \tilde{Y}_i) + \mathfrak{R}_\mu^{\tilde{\mathcal{D}}}(\mathcal{F}) + M \sqrt{\frac{\log(2/\delta')}{2\mu}}. \quad (17)$$

where $\delta' = \delta - 4(\mu - 1)\beta(a)$, and $\mathfrak{R}_\mu^{\tilde{\mathcal{D}}}(\mathcal{F})$ is the empirical Rademacher complexity over μ independent blocks. Assume the loss function ℓ is ξ -Lipschitz in its first argument, i.e.,

$$|\ell(y_1, y) - \ell(y_2, y)| \leq \xi \cdot \|y_1 - y_2\|, \quad \forall y_1, y_2, y \in \mathcal{Y}.$$

Let $f_h(x, y) := \ell(h(x), y)$ for $h \in \mathcal{H}$. Then, by the contraction lemma (Lemma 5.7 in [34]), we obtain

$$\mathfrak{R}_\mu^{\tilde{\mathcal{D}}}(\mathcal{F}) \leq \xi \cdot \mathfrak{R}_\mu^{\tilde{\mathcal{D}}}(\mathcal{H}'),$$

where $\mathcal{H}' := \{x \mapsto h(x) \in \mathcal{Y} \mid h \in \mathcal{H}\}$. Now consider that each $h \in \mathcal{H}$ is composed of L Transformer layers. Let $\mathcal{H}_s^l$ be the hypothesis class of the l -th layer. Then

$$h = f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(1)}, \quad \text{with } f^{(l)} \in \mathcal{H}_s^l.$$

By norm-based chaining arguments (see [12, 36]), the Rademacher complexity of $\mathcal{H}'$ can be bounded by a function G over the layer-wise complexities:

$$\mathfrak{R}_{\mu}^{\tilde{\mathcal{D}}}(\mathcal{H}') \leq G([\mathfrak{R}_{\mu}(\mathcal{H}_s^1)], \dots, [\mathfrak{R}_{\mu}(\mathcal{H}_s^L)]).$$

Substituting into Eq.(17), we have

$$\mathbb{E}_{(X,Y)}[\ell(h(X), Y)] \leq \frac{1}{\mu} \sum_{i=1}^{\mu} \ell(h(\tilde{X}_i), \tilde{Y}_i) + \xi \cdot G([\mathfrak{R}_{\mu}(\mathcal{H}_s^1)], \dots, [\mathfrak{R}_{\mu}(\mathcal{H}_s^L)]) + M \sqrt{\frac{\log(2/\delta')}{2\mu}}. \quad (18)$$

Finally, since $\tilde{\mathcal{D}}$ is sampled from $\mathcal{D}$ and blocks are representative under proper mixing, the empirical average can be approximated by the full dataset:

$$\frac{1}{\mu} \sum_{i=1}^{\mu} \ell(h(\tilde{X}_i), \tilde{Y}_i) \approx \frac{1}{m} \sum_{i=1}^m \ell(h(X_i), Y_i).$$

Substituting into Eq.18 gives the desired bound:

$$\mathbb{E}_{(X,Y)}[\ell(h(X), Y)] \leq \frac{1}{m} \sum_{i=1}^m \ell(h(X_i), Y_i) + \xi \cdot G([\mathfrak{R}_{\mu}(\mathcal{H}_s^1)], \dots, [\mathfrak{R}_{\mu}(\mathcal{H}_s^L)]) + M \sqrt{\frac{\log(2/\delta')}{2\mu}}.$$

□

H.2. Proof of Theorem 3.7

Proof. Let N_t denote the number of input tokens and d the hidden dimension of each token embedding. For layer indices $l \in \{0, \dots, L\}$ write $\mathcal{T}_k^l \in \mathbb{R}^d$ for the representation of token k after the l -th Transformer layer.

Firstly, define the input subspace

$$\mathcal{M}_0 := \text{span}\{\mathcal{T}_1^0, \dots, \mathcal{T}_{N_t}^0\}, \quad (19)$$

and let P_0 be the orthogonal projector onto $\mathcal{M}_0$. For every $i \geq 1$ and token k decompose

$$\mathcal{T}_k^i = V_k^i + U_k^i, \quad (20)$$

with $V_k^i := P_0 \mathcal{T}_k^i \in \mathcal{M}_0$ and $U_k^i := (I - P_0) \mathcal{T}_k^i \perp \mathcal{M}_0$. The drift coefficient is

$$\kappa_i := \frac{\sum_{k=1}^{N_t} \|U_k^i\|_2^2}{\sum_{k=1}^{N_t} \|\mathcal{T}_k^i\|_2^2} = 1 - \rho_i \in [0, 1], \quad (21)$$

where ρ_i is the topology-preservation ratio.

Secondly, let $\mathcal{D} = \{z_1, \dots, z_m\}$ be a fixed set of samples with m , and $z_k := \mathcal{T}_{j(k)}^{i-1}$. Let $\sigma_1, \dots, \sigma_m$ be i.i.d. Rademacher variables. Define

$$\mathfrak{R}(\mathcal{H}_s^i) := \frac{1}{m} \mathbb{E}_\sigma \left[\sup_{f \in \mathcal{H}_s^i} \sum_{k=1}^m \sigma_k f(z_k) \right]. \quad (22)$$

Thirdly, set $z'_k := V_{j(k)}^i$. Split

$$\sum_{k=1}^m \sigma_k f(z_k) = \sum_{k=1}^m \sigma_k f(z'_k) + \sum_{k=1}^m \sigma_k [f(z_k) - f(z'_k)]. \quad (23)$$

Aligned part:

$$\frac{1}{m} \sup_{f \in \mathcal{H}_s^i} \sum_{k=1}^m \sigma_k f(z'_k) \leq L_i \mathfrak{R}(\mathcal{H}_s^{i-1}). \quad (24)$$

Orthogonal part uses $|f(z_k) - f(z'_k)| \leq L_i \|U_{j(k)}^i\|_2$:

$$\frac{1}{m} \sup_{f \in \mathcal{H}_s^i} \sum_{k=1}^m |f(z_k) - f(z'_k)| \leq L_i \sqrt{\kappa_i} C_i, \quad (25)$$

where

$$C_i := \sqrt{\frac{1}{m} \sum_{k=1}^m \|\mathcal{T}_{j(k)}^i\|_2^2}. \quad (26)$$

Khintchine's inequality gives

$$C_i \leq 2 \mathfrak{R}(\mathcal{H}_s^{i-1}), \quad m \geq 1. \quad (27)$$

Combining Eq.(24)–(27) produces

$$\mathfrak{R}(\mathcal{H}_s^i) \leq L_i \sqrt{1 - \rho_i} \mathfrak{R}(\mathcal{H}_s^{i-1}). \quad (28)$$

Fourthly, iterate Eq.(28) from $i = 1$ to L , yields:

$$\mathfrak{R}(\mathcal{H}_s^L) \leq \mathfrak{R}(\mathcal{H}_s^0) \prod_{i=1}^L L_i \sqrt{1 - \rho_i}. \quad (29)$$

Because

$$\sqrt{1 - \rho_i} = \exp\left\{\frac{1}{2} \ln(1 - \rho_i)\right\}, \quad (30)$$

the bound can be written as

$$\mathfrak{R}(\mathcal{H}_s^L) \leq \mathfrak{R}(\mathcal{H}_s^0) \exp\left\{\frac{1}{2} \sum_{i=1}^L [\ln L_i + \ln(1 - \rho_i)]\right\}. \quad (31)$$

This is exactly Eq.(7), completing the proof. $\square$

I. Empirical Analyses

I.1. Weakening of Preservation Positional/Semantic Topology of the Input Tokens from an Information-Theoretic Perspective

From an information-theoretic perspective [5], the raw information contained in the input progressively weakens as network depth increases. This occurs because each layer in the network applies nonlinear transformations and compression to the input, resulting in the loss of information from the original input. Moreover, because positional/semantic topology of the input tokens is typically injected only once at the input layer, this information is gradually diluted or overridden in subsequent layers. Consequently, the model's preservation of positional/semantic topology of the input tokens weakens with the deepening of network layers.

I.2. Hilbert-Schmidt Independence Criterion (HSIC)

The Hilbert-Schmidt Independence Criterion (HSIC) is a kernel-based, non-parametric measure of statistical dependence between two variables [11, 47, 39]. By embedding the variables into reproducing-kernel Hilbert spaces (RKHSs) and evaluating the Hilbert-Schmidt norm of their cross-covariance operator, HSIC captures both linear and nonlinear relationships without requiring explicit density estimation.

HSIC (Hilbert-Schmidt Independence Criterion) theoretically requires the input to consist of two random variables. However, sinusoidal position encoding (PE) does not appear to meet this requirement, as it is deterministically generated by a fixed formula. In practice, though, HSIC relies solely on observed sample pairs (x_i, y_i) , and does not require these samples to be drawn from a truly random sampling process (see [39]). Specifically, given a set of token representations $H \in \mathbb{R}^{N_t \times D}$ and their corresponding deterministic sinusoidal position encodings $PE \in \mathbb{R}^{N_t \times D}$, we treat H and PE as collections of N_t variables. Each token and its position encoding at the same index form a sample pair (H^i, PE^i) , where $i = 1, \dots, N_t$. HSIC focuses on the dependency patterns among these sample pairs rather than on their generative process (see [39]). Therefore, using HSIC to verify whether token representations contain the positional information encoded by the sinusoidal PE remains reasonable and valid. Similarly, when evaluating the dependency between learnable position encodings (PE) and token features, or between the similarity matrix of input tokens and that of intermediate layer token representations, we can construct sample pairs based on corresponding positions or corresponding pairs. These sample pairs naturally satisfy the structural requirements for HSIC estimation, making it a suitable tool for assessing whether the model captures certain structural information or adjacency relations explicitly or implicitly.

Computation. Given two variables $X \in \mathbb{R}^{N_t \times D}$ and $Y \in \mathbb{R}^{N_t \times D}$, they can be decomposed to sample set: $\{(x_i, y_i)\}_{i=1}^{N_t}$, where $x_i, y_i \in \mathbb{R}^{1 \times D}$. For a sample (x_i, y_i) , define the kernel matrices $K^k \in \mathbb{R}^{N_t \times N_t}$ ($K_{ij}^k = \mathfrak{k}^k(x_i, x_j)$) and $L^k \in \mathbb{R}^{N_t \times N_t}$ ($L_{ij}^k = \mathfrak{l}^k(y_i, y_j)$), and the centering matrix $H_{center} = I_{N_t} - \frac{1}{N_t} \mathbf{1}\mathbf{1}^\top$. $\mathfrak{k}^k(x_i, x_j)$ and $\mathfrak{l}^k(y_i, y_j)$ are the Gaussian kernel functions which are expressed as:

$$\mathfrak{k}^k(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|_2^2}{2\sigma_x^2}\right), \mathfrak{l}^k(\mathbf{y}_i, \mathbf{y}_j) = \exp\left(-\frac{\|\mathbf{y}_i - \mathbf{y}_j\|_2^2}{2\sigma_y^2}\right) \quad (32)$$

where σ_x, σ_y are computed by the median heuristic. Specifically, first, given the sample matrix $X \in \mathbb{R}^{N \times D}$, evaluate the squared Euclidean distance for every unordered pair of samples

$$d_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\|_2^2, \quad i < j. \quad (33)$$

Second, collect all non-zero d_{ij} values and take their median, denoted by v_m . Finally, set the bandwidth as

$$\sigma_x = \sqrt{\frac{v_m}{2}}. \quad (34)$$

σ_y is obtained in the same manner. After getting the above items, the HSIC of two variables is computed as follows:

$$\text{HSIC}(X, Y) = \frac{1}{(N_t - 1)^2} \text{tr}(K^k H_{center} L^k H_{center}), \quad (35)$$

Table 10

Full results for the TSF task. The input length for all baseline models is set to 96, and the prediction lengths include {12, 24, 48, 96}. The results of our method are averaged over five random seeds, and the standard deviation across the five runs is reported after the “±” symbol.

Models	PatchTST+TEM (Ours)		PatchTST (2023)		iTransformer+TEM (Ours)		iTransformer (2024)		Transformer+TEM (Ours)		Transformer (2017)		Crossformer (2023)		TimesNet (2023)		DLinear (2023)		TIDE (2023)		
	Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
PEMS03	12	0.095±0.001	0.207±0.001	0.099	0.216	0.067±0.001	0.165±0.001	0.071	0.174	0.133±0.002	0.290±0.001	0.139	0.302	0.090	0.203	0.085	0.192	0.122	0.243	0.178	0.305
	24	0.136±0.000	0.249±0.001	0.142	0.259	0.088±0.001	0.191±0.000	0.093	0.201	0.191±0.001	0.348±0.001	0.199	0.363	0.121	0.240	0.118	0.223	0.201	0.317	0.257	0.371
	48	0.203±0.001	0.306±0.001	0.211	0.319	0.119±0.000	0.224±0.001	0.125	0.236	0.283±0.002	0.429±0.001	0.295	0.447	0.202	0.317	0.155	0.260	0.333	0.425	0.379	0.463
	96	0.258±0.001	0.355±0.002	0.269	0.370	0.156±0.001	0.261±0.001	0.164	0.275	0.362±0.002	0.497±0.002	0.377	0.518	0.262	0.367	0.228	0.317	0.457	0.515	0.490	0.539
	Avg	0.173	0.279	0.180	0.291	0.107	0.210	0.113	0.221	0.242	0.391	0.252	0.408	0.169	0.281	0.147	0.248	0.278	0.375	0.326	0.419
PEMS04	12	0.101±0.001	0.215±0.001	0.105	0.224	0.074±0.000	0.174±0.001	0.078	0.183	0.141±0.002	0.301±0.002	0.147	0.314	0.098	0.218	0.087	0.195	0.148	0.272	0.219	0.340
	24	0.147±0.000	0.264±0.001	0.153	0.275	0.090±0.001	0.195±0.000	0.095	0.205	0.205±0.002	0.370±0.001	0.214	0.385	0.131	0.256	0.103	0.215	0.224	0.340	0.292	0.398
	48	0.220±0.001	0.325±0.001	0.229	0.339	0.114±0.000	0.221±0.001	0.120	0.233	0.308±0.003	0.456±0.002	0.321	0.475	0.205	0.326	0.136	0.250	0.355	0.437	0.409	0.478
	96	0.279±0.001	0.373±0.002	0.291	0.389	0.142±0.001	0.249±0.001	0.150	0.262	0.391±0.002	0.523±0.002	0.407	0.545	0.402	0.457	0.190	0.303	0.452	0.504	0.492	0.532
	Avg	0.187	0.295	0.195	0.307	0.105	0.210	0.111	0.221	0.261	0.412	0.272	0.430	0.209	0.314	0.129	0.241	0.295	0.388	0.353	0.437
PEMS07	12	0.091±0.001	0.199±0.001	0.095	0.207	0.064±0.001	0.157±0.000	0.067	0.165	0.128±0.002	0.278±0.001	0.133	0.290	0.094	0.200	0.082	0.181	0.115	0.242	0.173	0.304
	24	0.144±0.000	0.252±0.001	0.150	0.262	0.084±0.001	0.180±0.001	0.088	0.190	0.202±0.002	0.352±0.002	0.210	0.367	0.139	0.247	0.101	0.204	0.210	0.329	0.271	0.383
	48	0.243±0.001	0.326±0.001	0.253	0.340	0.104±0.000	0.204±0.001	0.110	0.215	0.340±0.002	0.457±0.002	0.354	0.476	0.311	0.369	0.134	0.238	0.398	0.458	0.446	0.495
	96	0.332±0.001	0.388±0.001	0.346	0.404	0.132±0.001	0.233±0.001	0.139	0.245	0.465±0.003	0.543±0.002	0.484	0.566	0.396	0.442	0.181	0.279	0.594	0.553	0.628	0.577
	Avg	0.203	0.291	0.211	0.303	0.096	0.194	0.101	0.204	0.284	0.408	0.295	0.425	0.235	0.315	0.124	0.225	0.329	0.395	0.380	0.440
PEMS08	12	0.161±0.001	0.223±0.001	0.168	0.232	0.075±0.000	0.173±0.001	0.079	0.182	0.226±0.002	0.312±0.002	0.235	0.325	0.165	0.214	0.112	0.212	0.154	0.276	0.227	0.343
	24	0.215±0.001	0.270±0.001	0.224	0.281	0.109±0.001	0.208±0.001	0.115	0.219	0.301±0.002	0.377±0.002	0.314	0.393	0.215	0.260	0.141	0.238	0.248	0.353	0.318	0.409
	48	0.308±0.001	0.340±0.001	0.321	0.354	0.177±0.001	0.223±0.001	0.186	0.235	0.431±0.002	0.476±0.002	0.449	0.496	0.315	0.355	0.198	0.283	0.440	0.470	0.497	0.510
	96	0.392±0.001	0.400±0.002	0.408	0.417	0.210±0.001	0.254±0.001	0.221	0.267	0.548±0.003	0.561±0.002	0.571	0.584	0.377	0.397	0.320	0.351	0.674	0.565	0.721	0.592
	Avg	0.269	0.308	0.280	0.321	0.142	0.215	0.150	0.226	0.376	0.432	0.392	0.450	0.268	0.307	0.193	0.271	0.379	0.416	0.441	0.464

Table 11

The performance of applying sinusoidal PE to iTransformer on the Weather dataset. The lookback length for all models is set to 96, and the prediction lengths ∈ {96, 192, 336, 720}.

MSE/MAE	Dataset	Prediction length	Original	+sinusoidal PE
iTransformer	Weather	96	0.174/0.214	0.172/0.212
		192	0.221/0.254	0.219/0.257
		336	0.278/0.296	0.276/0.295
		720	0.358/0.349	0.356/0.347
		Avg	0.258/0.279	0.256/0.278

where $\text{tr}(\cdot)$ denotes the matrix trace. Equation (35) involves only matrix multiplications and traces, making HSIC straightforward to implement and scalable with modern hardware.

I.3. More Forecasting Results

Due to space constraints in the main text, this section presents the more multivariate forecasting results. We conducted an extensive evaluation of several competitive baselines on challenging forecasting tasks, and the outcomes are summarized in Table 10. All reported figures of our methods are the averages over five random seeds. As the table shows, integrating our TEM into various baseline models consistently yields improved forecasting performance.

I.4. The Importance of Position Encoding for Variable Tokens

iTransformer considers the PE of variable tokens unimportant [30], which is an unreasonable viewpoint. Although swapping the positions of variable tokens does not alter the semantics of the input sequence (i.e., permutation invariance), making the relative positions of variable tokens seemingly meaningless, the absolute positions of variable tokens still play a crucial role. Through the absolute positions, the model can directly recognize variables without needing to identify them indirectly through features. We believe that the PE of variable tokens remains meaningful. Experimental results (Table 11) also validate this perspective: on the Weather dataset, adding sinusoidal PE to iTransformer and retraining it significantly reduces MSE/MAE. Furthermore, other methods that utilize variable tokens, such as Crossformer [56], also incorporate PE for variable tokens.

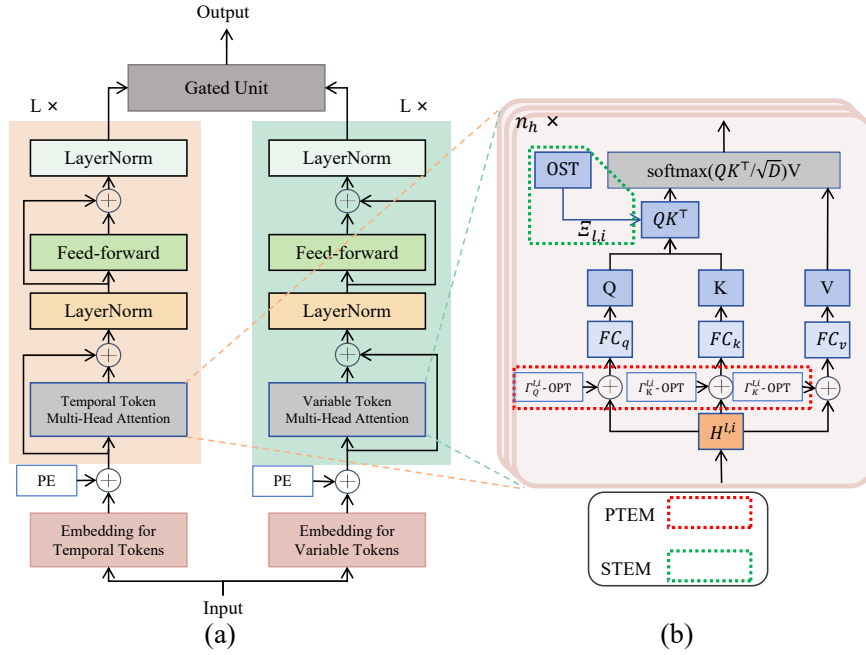


Figure 17: Overview of the Concise Dual-branch Transformer Framework (CDTF). (a) The structure of CDTF. (b) Illustration of the integration locations of PTEM and STEM within CDTF.

J. Further Work: Applying TEM to a Dual-Branch Framework

In the main text, we apply our TEM to three single-branch Transformer-based time prediction methods (PatchTST [37], iTransformer [30], Transformer [45]). To further verify the generality of TEM, we apply it to a dual-branch Transformer-based time prediction method in this section. However, current dual-branch methods, for the sake of computational efficiency, either reduce the number of tokens in the output series as the network deepens [56], or replace the attention mechanism with more efficient convolution operations [6, 17]. In the former work [56], the similarity matrix of the initial tokens has a shape inconsistent with the intermediate QK^T in the network, making direct addition infeasible and thus preventing the injection of OST via Equation 9. In the latter work [6], QK^T is absent altogether, similarly making it impossible to inject OST through Equation 9. These approaches make it infeasible to apply STEM to them.

To address the aforementioned issues, we design a Concise Dual-branch Transformer Framework (CDTF) to incorporate our TEM method, aiming to validate TEM's applicability and effectiveness within a dual-branch architecture. The framework is straightforward, with each branch implemented as a basic Transformer encoder: the left branch processes temporal tokens, while the right branch handles variable tokens. In this section, we first introduce the architectural details of the model (see Appendix J.1), followed by implementation details (see Appendix J.2), and finally present comparative results with existing methods as well as ablation studies (see Appendix J.3).

J.1. A Concise Dual-branch Transformer Framework with our TEM

In this subsection, we present our Concise Dual-branch Transformer Framework (CDTF), which leverages temporal tokens and variable tokens to boost model performance. CDTF first processes the correlations between temporal tokens and the correlations between variable tokens in two separate branches, each built on the original Transformer encoder. It then merges the features from both branches through a gated unit to make predictions. An overview of CDTF is shown in Fig. 17. Next, we detail the temporal-token branch, the variable-token branch, and the fusion module (the gating unit) in turn.

Temporal token branch: This branch aims to handle the correlations between the temporal tokens. Specifically, we get enhanced input features ($H_Q^{l-1,i}$, $H_K^{l-1,i}$, $H_V^{l-1,i}$) of the l -th layer of this branch through Eq.(8). Then, we calculate

the Q , K , and V of the i -th attention head in the l -th layer of this branch as follows:

$$\begin{aligned} Q_{l,i}^{time} &= \mathbf{FC}_q^{time}(H_Q^{l-1,i}) \in \mathbb{R}^{T \times D}, K_{l,i}^{time} = \mathbf{FC}_k^{time}(H_K^{l-1,i}) \in \mathbb{R}^{T \times D}, \\ V_{l,i}^{time} &= \mathbf{FC}_v^{time}(H_V^{l-1,i}) \in \mathbb{R}^{T \times D}, l = 1, \dots, L, i = 1, \dots, n_t \end{aligned} \quad (36)$$

where $\mathbf{FC}_q^{time}$, $\mathbf{FC}_k^{time}$ and $\mathbf{FC}_v^{time}$ are three fully connected layers, L is the number of encoder layers, n_t is the number of head. After obtaining Q , K , and V , according to Eq.(9), we calculate the attention map of the i -th attention head in the l -th layer of this branch as follows:

$$A_{l,i} = \text{softmax}\left(\frac{Q_{l,i}^{time}[K_{l,i}^{time}]^T + \Xi_{l,i} \times S^0}{\sqrt{D}}\right), l = 1, \dots, L, i = 1, \dots, n_t. \quad (37)$$

The rest of the computation process remains consistent with the vanilla Transformer encoder. Firstly, we use attention mechanism to produce the refined feature $A_{l,i}V_{l,i}^{time}$, then we fuse the multi-head output, next the fused feature goes through a feedforward network (FFN) to obtain the output of the current layer, recorded as H_l^{time} which serves as the input embedding for the next layer. This process is repeated L times throughout the entire model. Finally, we obtain the output features of the temporal tokens branch, i.e., $H_{out}^{time} \in \mathbb{R}^{T \times D}$.

Variable token branch: This branch aims to handle the correlations between variable tokens. The computation process is the same as the temporal token branch. The only difference is that this branch processes variable tokens with shape $N \times D$. Finally, we obtain the output features of the variable tokens branch, i.e., $H_{out}^{var} \in \mathbb{R}^{N \times D}$.

Fusion Module: In this module, we aim to merge the outputs of the above two branches. We propose a gated unit specially designed for TSF to achieve this goal. The gated unit dynamically assesses the two branches' importance based on their outputs and performs weighted fusion. Specifically, we first use three fully connected layers to reshape the output features from both branches to the same shape:

$$F_t = \mathbf{FC}_{t2}[\mathbf{FC}_{t1}(H_{out}^{time})]^T \in \mathbb{R}^{N \times S}, F_v = \mathbf{FC}_v(H_{out}^{var}) \in \mathbb{R}^{N \times S}, \quad (38)$$

where $\mathbf{FC}_{t1} : \mathbb{R}^D \rightarrow \mathbb{R}^N$, $\mathbf{FC}_{t2} : \mathbb{R}^T \rightarrow \mathbb{R}^S$, $\mathbf{FC}_v : \mathbb{R}^D \rightarrow \mathbb{R}^S$ are three fully connected layers, F_t is the refined feature of the temporal token branch, F_v is the refined features of the variable token branch. Next, we employ the gated unit to fuse F_t and F_v :

$$G = \sigma([F_t, F_v]W_f) \in \mathbb{R}^{N \times S}, \hat{Y} = G \odot F_v + (1 - G) \odot F_t, \quad (39)$$

where $[\dots]$ denotes the operation of concatenation, $[F_t, F_v] \in \mathbb{R}^{N \times 2S}$, $W_f \in \mathbb{R}^{2S \times S}$, σ denotes a sigmoid operation, $\odot$ denotes element-wise multiplication and $\hat{Y}^T \in \mathbb{R}^{S \times N}$ is the final output. For simplicity, we let $\hat{Y} = \hat{Y}^T$.

We use the MSE loss and the bi-level optimization strategy mentioned in Section 4.3 to update the model.

J.2. Implementation Details

Data undergo mean-std normalization before input into the model and inverse mean-std normalization before output. The proposed models are trained using an NVIDIA 4090 GPU. MSE and MAE are employed as evaluation metrics. In our experiments, we set the number of layers of the encoder L to 2 for ETT datasets, 3 for the Weather dataset, and 4 for other datasets. The token dimension D is set to 128 for the ETT dataset and 512 for other datasets. The head number of attention is set to 8. The batch size is set to 16 for the ECL and Traffic datasets and 32 for others. For the bi-level optimization, in the inner loop, we adopt the Adam optimizer [22] with a decaying learning rate to update the backbone parameters. The initial learning rate is shown in Table 12. In the outer loop, the positional/semantic topology weights are updated using the Adam optimizer with a decaying learning rate. The initial learning rate is set to $1e-3$. The total number of epochs is 40. The training process will stop early if the validation loss doesn't decrease within three epochs.

J.3. Experiment Results

Comparative Experimental Results Table 13 and Table 14 present the comprehensive prediction results. The best results are highlighted in red, while the second-best are marked in blue. Lower MSE/MAE values indicate higher prediction accuracy. Compared with other time series forecasting (TSF) models, our method achieves the best average performance on most benchmark datasets. These results demonstrate the superior performance of our proposed

Table 12

Hyperparameter setting of CDTF for different datasets.

Dataset	Prediction Length	lr_{init}
ETTh1	96	9.2e-3
	192	1.33e-2
	336	3e-4
	720	1e-4
ETTh2	96	2.9e-3
	192	5e-3
	336	7e-3
	720	7e-3
ETTm1	96	1.08e-2
	192	6.2e-3
	336	9.8e-3
	720	4.5e-3
ETTm2	96	1.1e-3
	192	1e-4
	336	2e-3
	720	6.6e-3
ECL	96	9.05e-4
	192	1.08e-3
	336	1.38e-3
	720	9.2e-4
Traffic	96	1.07e-3
	192	1.01e-3
	336	9.2e-4
	720	1.22e-3
Weather	96	1.3e-3
	192	9.1e-5
	336	7e-4
	720	7e-4
Solar-Energy	96	1.6e-4
	192	1.6e-4
	336	1.6e-4
	720	1.6e-4

approach. Furthermore, the encoder of each branch of our model is identical to the original Transformer encoder, with the only difference being the introduction of the PTEM and STEM modules. The experimental results show that even with a minimal dual-branch Transformer structure, integrating our proposed modules can achieve competitive performance on most datasets. These findings further validate the effectiveness of our method.

Ablation Study of PTEM, STEM and Dual-branch In this subsection, we conduct a systematic, quantitative study of the performance gains contributed by PTEM, STEM, and the dual-branch architecture. Specifically, we independently remove PTEM, STEM, PTEM&STEM, the temporal branch, and the variable branch. Experiments are performed on the ETTm1, Weather, and ECL datasets, which contain 7, 21, and 321 variables, respectively, representing three typical scenarios with small, medium, and large numbers of variables. The results are shown in Table 15. From the results, we can draw three conclusions. (1) Using the original algorithm (Original) and the dual-branch framework (corresponding “-PTEM&STEM” column) as reference points, we find that adding PTEM alone to the dual-branch framework (corresponding “-STEM” column) accounts for about 61.48% of the gain achieved when both PTEM and STEM are added simultaneously (Original). Adding STEM alone (corresponding “-PTEM” column) accounts for about 56.71% of that gain. (2) Taking the pure variable-branch algorithm (corresponding “-Temporal Branch&PTEM&STEM” column) as the baseline. Adding the temporal branch to it (corresponding “-PTEM & STEM” column) lowers the MSE by 1.72%, 1.55%, and 1.12% on ETTm1, Weather, and ECL, respectively. When PTEM & STEM are further incorporated into this dual-branch framework (Original), the MSE reductions increase to 4.67%, 6.2%, and 7.30% on the same datasets. Hence, the dual-branch design itself contributes 36.84%, 25%, and 15.34% of the total MSE reduction on ETTm1 (7 variables), Weather (21 variables), and ECL (321 variables), respectively. This pattern suggests that the fewer the variables, the more pronounced the benefit of adding the temporal branch, while PTEM & STEM remain the dominant source of performance improvement overall. (3) The columns “-Temporal Branch” and “-Variable Branch” reveal that the temporal branch alone performs worse than the variable

branch. Nevertheless, in settings with scarce variables (ETTm1), the temporal branch provides valuable supplementary information, making its inclusion practically necessary.

Table 13

Full MSE results for the TSF task. All baseline models' input length is 96, and the prediction lengths include {96, 192, 336, 720}. The results of our method are averaged over five random seeds, and the standard deviation across the five runs is reported after the “ $\pm$ ” symbol.

Models		CDTF (Ours)	PatchTST (2023)	iTransformer (2024)	Crossformer (2023)	TimesNet (2023)	DLinear (2023)	DSformer (2023)	Transformer (2017)	SpareTSF (2024)	PRReg CI (2024)	TIDE (2023)
Metric		MSE ($\downarrow$)										
ETTh1	96	0.382 $\pm$ 0.002	0.414	0.386	0.423	0.384	0.386	0.401	0.611	0.374	0.409	0.479
	192	0.436 $\pm$ 0.003	0.460	0.441	0.471	0.436	0.437	0.441	0.697	0.419	0.454	0.525
	336	0.476 $\pm$ 0.002	0.501	0.487	0.570	0.491	0.481	0.486	0.770	0.463	0.514	0.565
	720	0.486 $\pm$ 0.001	0.500	0.503	0.653	0.521	0.519	0.493	0.796	0.477	0.530	0.594
	Avg	0.445	0.469	0.454	0.529	0.458	0.456	0.455	0.718	0.433	0.477	0.541
ETT2	96	0.289 $\pm$ 0.001	0.302	0.297	0.745	0.340	0.333	0.319	0.470	0.308	0.300	0.400
	192	0.377 $\pm$ 0.001	0.388	0.380	0.877	0.402	0.477	0.400	0.601	0.388	0.385	0.528
	336	0.410 $\pm$ 0.000	0.426	0.428	1.043	0.452	0.594	0.462	0.676	0.431	0.440	0.643
	720	0.411 $\pm$ 0.002	0.431	0.427	1.104	0.462	0.831	0.457	0.676	0.432	0.426	0.874
	Avg	0.372	0.387	0.383	0.942	0.414	0.559	0.410	0.605	0.390	0.388	0.611
ETTm1	96	0.317 $\pm$ 0.002	0.329	0.334	0.404	0.338	0.345	0.342	0.528	0.349	0.344	0.364
	192	0.367 $\pm$ 0.002	0.367	0.377	0.450	0.374	0.380	0.385	0.596	0.385	0.384	0.398
	336	0.403 $\pm$ 0.003	0.399	0.426	0.532	0.410	0.413	0.402	0.674	0.412	0.442	0.428
	720	0.466 $\pm$ 0.000	0.454	0.491	0.666	0.478	0.474	0.472	0.777	0.474	0.479	0.487
	Avg	0.388	0.387	0.407	0.513	0.400	0.403	0.400	0.643	0.405	0.412	0.419
ETTm2	96	0.172 $\pm$ 0.001	0.175	0.180	0.287	0.187	0.193	0.184	0.285	0.180	0.187	0.207
	192	0.238 $\pm$ 0.002	0.241	0.250	0.414	0.249	0.284	0.248	0.396	0.243	0.253	0.290
	336	0.305 $\pm$ 0.002	0.305	0.311	0.597	0.321	0.369	0.335	0.492	0.302	0.320	0.377
	720	0.405 $\pm$ 0.003	0.402	0.412	1.730	0.408	0.554	0.412	0.652	0.398	0.440	0.558
	Avg	0.280	0.281	0.288	0.757	0.291	0.350	0.295	0.456	0.281	0.300	0.358
ECL	96	0.136 $\pm$ 0.001	0.195	0.148	0.219	0.168	0.197	0.173	0.260	0.194	0.146	0.237
	192	0.154 $\pm$ 0.000	0.199	0.162	0.231	0.184	0.196	0.183	0.266	0.197	0.160	0.236
	336	0.168 $\pm$ 0.001	0.215	0.178	0.246	0.198	0.209	0.203	0.280	0.209	0.172	0.249
	720	0.203 $\pm$ 0.004	0.256	0.225	0.280	0.220	0.245	0.259	0.302	0.251	0.209	0.284
	Avg	0.165	0.216	0.178	0.244	0.192	0.212	0.205	0.277	0.213	0.172	0.251
Traffic	96	0.384 $\pm$ 0.003	0.544	0.395	0.522	0.593	0.650	0.529	0.647	0.543	0.403	0.805
	192	0.399 $\pm$ 0.002	0.540	0.417	0.530	0.617	0.598	0.533	0.649	0.555	0.416	0.756
	336	0.418 $\pm$ 0.002	0.551	0.433	0.558	0.629	0.605	0.545	0.667	0.565	0.448	0.762
	720	0.456 $\pm$ 0.004	0.586	0.467	0.589	0.640	0.645	0.583	0.697	0.639	0.471	0.719
	Avg	0.414	0.555	0.428	0.550	0.620	0.625	0.548	0.665	0.576	0.435	0.760
Weather	96	0.156 $\pm$ 0.001	0.177	0.174	0.158	0.172	0.196	0.162	0.395	0.180	0.171	0.202
	192	0.206 $\pm$ 0.001	0.225	0.221	0.206	0.219	0.237	0.211	0.619	0.226	0.219	0.242
	336	0.263 $\pm$ 0.001	0.278	0.278	0.272	0.280	0.283	0.267	0.689	0.281	0.286	0.287
	720	0.344 $\pm$ 0.000	0.354	0.358	0.398	0.365	0.345	0.343	0.926	0.358	0.357	0.351
	Avg	0.242	0.259	0.258	0.259	0.259	0.265	0.246	0.657	0.261	0.258	0.271
Solar-Energy	96	0.197 $\pm$ 0.002	0.234	0.203	0.310	0.250	0.290	0.247	0.386	0.245	0.204	0.312
	192	0.228 $\pm$ 0.002	0.267	0.233	0.734	0.296	0.320	0.288	0.444	0.255	0.249	0.339
	336	0.242 $\pm$ 0.002	0.290	0.248	0.750	0.319	0.353	0.329	0.471	0.259	0.260	0.368
	720	0.248 $\pm$ 0.003	0.289	0.249	0.769	0.338	0.356	0.341	0.473	0.262	0.271	0.370
	Avg	0.229	0.270	0.233	0.641	0.301	0.330	0.301	0.442	0.255	0.246	0.347

Table 14

Full MAE results for the TSF task. All baseline models' input length is 96, and the prediction lengths include {96, 192, 336, 720}. The results of our method are averaged over five random seeds, and the standard deviation across the five runs is reported after the “ $\pm$ ” symbol.

Models	CDTF (Ours)		PatchTST (2023)	iTransformer (2024)	Crossformer (2023)	TimesNet (2023)	DLinear (2023)	DSformer (2023)	Transformer (2017)	SpareTSF (2024)	PRReg CI (2024)	TIDE (2023)
Metric		MAE (↓)										
ETTh1	96	0.399±0.001	0.419	0.405	0.448	0.402	0.400	0.412	0.539	0.390	0.408	0.464
	192	0.431±0.002	0.445	0.436	0.474	0.429	0.432	0.435	0.580	0.420	0.447	0.492
	336	0.450±0.003	0.466	0.458	0.546	0.469	0.459	0.456	0.609	0.440	0.483	0.515
	720	0.477±0.003	0.488	0.491	0.621	0.500	0.516	0.475	0.653	0.472	0.490	0.558
	Avg	0.439	0.454	0.447	0.522	0.450	0.452	0.446	0.594	0.431	0.457	0.507
ETTh2	96	0.342±0.000	0.348	0.347	0.584	0.374	0.387	0.359	0.461	0.354	0.363	0.440
	192	0.400±0.001	0.400	0.400	0.656	0.414	0.476	0.410	0.532	0.398	0.414	0.509
	336	0.425±0.002	0.433	0.432	0.731	0.452	0.541	0.451	0.575	0.443	0.452	0.571
	720	0.437±0.001	0.446	0.445	0.763	0.468	0.657	0.463	0.592	0.447	0.466	0.679
	Avg	0.401	0.407	0.407	0.684	0.427	0.515	0.421	0.541	0.411	0.424	0.550
ETTm1	96	0.357±0.002	0.367	0.368	0.426	0.375	0.372	0.375	0.489	0.381	0.384	0.387
	192	0.386±0.002	0.385	0.391	0.451	0.387	0.389	0.392	0.520	0.397	0.399	0.404
	336	0.407±0.002	0.410	0.420	0.515	0.411	0.413	0.411	0.559	0.417	0.426	0.425
	720	0.443±0.003	0.439	0.459	0.589	0.450	0.453	0.448	0.610	0.450	0.455	0.461
	Avg	0.398	0.400	0.410	0.496	0.406	0.407	0.407	0.545	0.411	0.416	0.419
ETTm2	96	0.258±0.001	0.259	0.264	0.366	0.267	0.292	0.269	0.351	0.259	0.278	0.305
	192	0.300±0.002	0.302	0.309	0.492	0.309	0.362	0.306	0.411	0.308	0.329	0.364
	336	0.344±0.002	0.343	0.348	0.542	0.351	0.427	0.365	0.463	0.342	0.366	0.422
	720	0.400±0.003	0.400	0.407	1.042	0.403	0.522	0.409	0.541	0.398	0.428	0.524
	Avg	0.326	0.326	0.332	0.610	0.333	0.401	0.337	0.442	0.327	0.350	0.404
ECL	96	0.233±0.000	0.285	0.240	0.314	0.272	0.282	0.269	0.358	0.269	0.239	0.329
	192	0.251±0.001	0.289	0.253	0.322	0.289	0.285	0.280	0.367	0.273	0.262	0.330
	336	0.266±0.002	0.305	0.269	0.337	0.300	0.301	0.297	0.375	0.287	0.276	0.344
	720	0.294±0.003	0.337	0.317	0.363	0.320	0.333	0.340	0.386	0.321	0.306	0.373
	Avg	0.261	0.304	0.270	0.334	0.295	0.300	0.297	0.372	0.288	0.271	0.344
Traffic	96	0.269±0.002	0.359	0.268	0.290	0.321	0.396	0.370	0.357	0.332	0.279	0.493
	192	0.275±0.002	0.354	0.276	0.293	0.336	0.370	0.366	0.356	0.356	0.301	0.474
	336	0.285±0.002	0.358	0.283	0.305	0.336	0.373	0.370	0.364	0.360	0.305	0.477
	720	0.299±0.003	0.375	0.302	0.328	0.350	0.394	0.386	0.376	0.352	0.322	0.449
	Avg	0.282	0.362	0.282	0.304	0.336	0.383	0.373	0.363	0.350	0.302	0.473
Weather	96	0.203±0.000	0.218	0.214	0.230	0.220	0.255	0.207	0.427	0.230	0.217	0.261
	192	0.251±0.002	0.259	0.254	0.277	0.261	0.296	0.252	0.560	0.271	0.270	0.298
	336	0.290±0.001	0.297	0.296	0.335	0.306	0.335	0.294	0.594	0.311	0.314	0.335
	720	0.341±0.002	0.348	0.349	0.418	0.359	0.381	0.343	0.710	0.360	0.349	0.386
	Avg	0.271	0.281	0.279	0.315	0.287	0.317	0.274	0.572	0.293	0.288	0.320
Solar-Energy	96	0.229±0.001	0.286	0.237	0.331	0.292	0.378	0.292	0.372	0.291	0.251	0.399
	192	0.260±0.002	0.310	0.261	0.725	0.318	0.398	0.320	0.409	0.294	0.272	0.416
	336	0.271±0.002	0.315	0.273	0.735	0.330	0.415	0.344	0.429	0.300	0.285	0.430
	720	0.276±0.002	0.317	0.275	0.765	0.337	0.413	0.352	0.432	0.303	0.292	0.425
	Avg	0.259	0.307	0.262	0.639	0.319	0.401	0.327	0.411	0.297	0.275	0.417

Table 15

Ablation study of PTEM, STEM, temporal branch, and variable branch. Look-back length is fixed at 96; prediction lengths $\in \{96, 192, 336, 720\}$.

Dataset	Pred Len	Original	-PTEM	-STEM	-PTEM & STEM	-Temporal Branch&PTEM & STEM	-Temporal Branch	-Variable Branch
ETT _{m1}	96	0.317 / 0.357	0.326 / 0.361	0.321 / 0.359	0.327 / 0.366	0.334 / 0.368	0.328 / 0.365	0.408 / 0.415
	192	0.367 / 0.386	0.372 / 0.387	0.371 / 0.388	0.374 / 0.390	0.377 / 0.391	0.374 / 0.390	0.461 / 0.441
	336	0.403 / 0.407	0.411 / 0.410	0.407 / 0.409	0.415 / 0.415	0.426 / 0.420	0.421 / 0.415	0.489 / 0.481
	720	0.466 / 0.443	0.472 / 0.446	0.473 / 0.445	0.482 / 0.452	0.491 / 0.459	0.485 / 0.456	0.542 / 0.512
	Avg	0.388 / 0.398	0.395 / 0.401	0.393 / 0.400	0.400 / 0.406	0.407 / 0.410	0.402 / 0.406	0.494 / 0.462
ECL	96	0.136 / 0.233	0.140 / 0.234	0.138 / 0.233	0.149 / 0.238	0.148 / 0.240	0.141 / 0.235	0.270 / 0.362
	192	0.154 / 0.251	0.156 / 0.253	0.157 / 0.252	0.160 / 0.254	0.162 / 0.253	0.156 / 0.252	0.282 / 0.374
	336	0.168 / 0.266	0.171 / 0.269	0.171 / 0.267	0.175 / 0.272	0.178 / 0.269	0.169 / 0.264	0.288 / 0.382
	720	0.203 / 0.294	0.207 / 0.297	0.204 / 0.304	0.221 / 0.315	0.225 / 0.317	0.209 / 0.305	0.291 / 0.387
	Avg	0.165 / 0.261	0.168 / 0.263	0.169 / 0.264	0.176 / 0.270	0.178 / 0.270	0.169 / 0.264	0.283 / 0.376
Weather	96	0.156 / 0.203	0.166 / 0.209	0.165 / 0.210	0.172/0.212	0.174 / 0.214	0.165 / 0.208	0.184 / 0.275
	192	0.206 / 0.251	0.212 / 0.253	0.210 / 0.252	0.219/0.254	0.221 / 0.254	0.210 / 0.252	0.225 / 0.311
	336	0.263 / 0.290	0.268 / 0.292	0.271 / 0.292	0.274/0.295	0.278 / 0.296	0.266 / 0.293	0.344 / 0.372
	720	0.344 / 0.341	0.348 / 0.344	0.347 / 0.346	0.351/0.346	0.358 / 0.349	0.350 / 0.345	0.480 / 0.498
	Avg	0.242 / 0.271	0.249 / 0.275	0.248 / 0.275	0.254/0.277	0.258 / 0.279	0.248 / 0.275	0.316 / 0.364